BEAST-MEV: Batched Threshold Encryption with Silent Setup for MEV prevention

Jan Bormet*, Arka Rai Choudhuri†, Sebastian Faust*, Sanjam Garg‡, Hussien Othman*, Guru-Vamsi Policharla§, Ziyan Qu*, Mingyuan Wang¶
*Technische Universität Darmstadt
†Nexus
‡UC Berkeley and Exponential Science Foundation
§UC Berkeley
¶NYU Shanghai

***Abstract*—Threshold encrypted mempools protect the privacy of transactions up until the point their inclusion on chain is confirmed. They are a promising approach to protection against front-running attacks on decentralized blockchains.**

Recent works have introduced two key properties that an encryption scheme must satisfy in order to scale to large scale decentralized blockchains such as Ethereum: Silent Setup [Garg-Kolonelos-Policharla-Wang, CRYPTO'24], demands that a threshold encryption scheme does not require any interaction during the setup phase and only relies on the existence of Public Key Infrastructure. Batched Decryption [Choudhuri-Garg-Piet-Policharla, USENIX'24], demands that an entire block containing B encrypted transactions can be decrypted using communication that is independent of (or sublinear in) B, without compromising the privacy of transactions that have not yet been confirmed.

While existing constructions achieve either Silent Setup or Batched Decryption independently, a truly decentralized and *scalable* encrypted mempool requires both properties to be satisfied simultaneously. In this work, we present the first "Batched Threshold Encryption scheme with Silent Setup" built using bilinear pairings. We provide formal definitions for the primitive, and prove security in the Generic Group Model. We provide several optimizations and implement our scheme to evaluate its performance. Our experiments demonstrate its efficiency for deployment in blockchain systems.

1. Introduction

One of the most significant applications of public blockchains is decentralized finance (DeFi), in which users access innovative financial products to generate trading returns. Over the past few years, the DeFi ecosystem has experienced explosive growth in both locked capital and product innovation. For instance, Ethereum alone currently holds over $60 billion in DeFi deposits, and several competing networks each boast multi-billion-dollar ecosystems of their own. While DeFi has several advantages in terms of decentralization, and being permissionless, it is prone to market manipulation attacks from front-running, sandwiching, and more [1], [2], [3], [4], [5], [6], [7], [8].

To illustrate these attacks, let us recall how blockchains process transactions that are submitted by users to the blockchain miners. The miners collect these unprocessed transactions in a so-called *mempool*. Once a miner is elected as a block proposer, it selects some non-finalized transactions from the mempool and assembles them into a block. Crucially, since miners can decide upon the order of the transactions within a block, a malicious miner can, e.g., place a profitable transaction before an honest users' trade (known as frontrunning). Such re-ordering can reap huge financial profits, which is often referred to as *miner/maximal extractable value (MEV)*. The presence of MEV opportunities can lead to centralization issues, e.g., currently the majority of Ethereum blocks are produced by just two block builders. [1] Since this can de-stabilize the

1. Notice that in current Ethereum, this is due to the proposer-builder separation where special parties, so-called builders, are responsible for pre-building blocks.

underlying consensus mechanism [9], developing countermeasures against MEV attacks has gained huge interest both by academia and industry [10], [11], [12], [13].

Among the proposed mitigations, one particularly promising approach is to keep transaction content private upon entry into the mempool. By concealing transaction details (e.g., the type of swap a user wants to execute), this effectively prevents miners from selecting and ordering transactions based on their content, thereby preventing MEV. However, for the system to remain usable, transactions must be available in the clear *after* they are finalized. The naïve implementation, in which clients commit their transactions and later return to reveal them if selected, is inadequate since it places a burden on clients to remain online and introduces the risk that clients may refuse to reveal transactions they no longer deem favorable.

Instead, an appealing cryptographic solution is to use threshold encryption to construct *private mempools* [11], [14], [15], [16], [17], [18], [19]. In this setting, during a setup phase a set of n servers (often called a *committee*) jointly generates a public encryption key that is published on a blockchain and a decryption key that is shared among the servers. Users can encrypt their transactions under the public key and send them to the miners, where they queue in the mempool for processing. Later, once the miners have selected B encrypted transactions from the mempool and built a block, the members of the decryption committee broadcast partial decryptions, enabling anyone in the system to decrypt and recover the corresponding transaction data of the B selected transactions. The security guarantee is that as long as fewer than $t < n$ servers are malicious, re-ordering based on transaction content is prevented.

For use in practice, two further properties are important: (i) since the partial decryptions are stored (at least temporarily) on the blockchain, the size of each server's partial decryption should be a constant, independent of the number of selected transactions B; and (ii) ciphertexts that are *not* selected should *remain private*, a notion formalized as *pending transaction privacy* in [16]. Schemes satisfying these criteria are referred to as *batched* threshold encryption schemes. While it is relatively easy to construct a scheme satisfying either of these properties in isolation, recent works have shown how to construct practical BTE schemes satisfying both properties (with various additional features) [16], [17], [18], [19].

One crucial bottleneck of current batched threshold encryption schemes (and in fact most threshold cryptosystems in general) is the requirement of an interactive setup. Indeed, all existing constructions of BTE schemes [16], [17], [18], [19] assume parties holding Shamir secret shares of the decryption key. In the setting of blockchains, where no trusted party exists, this would require all committee members to run an interactive setup protocol, known as the distributed key generation (DKG) protocol. Running DKG is highly undesirable in practice due to its costly concrete efficiency. Moreover, in a permissionless setting, the committee is typically refreshed on a regular basis, which means that such a costly DKG protocol needs to be repeated as well every time a new committee is selected. For our application of encrypted mempools, the costs of DKG are particularly problematic as ideally we would like all miners to participate in the decryption process. Since major blockchains such as Ethereum are run by hundreds of thousands of miners (or validators in case of Ethereum), the costs of running DKG become effectively prohibitive. What we really need is a *non-interactive* and *one-time* setup that enables the formation of all future committees, when *no complex on-boarding or off-boarding* is required when miners join or leave the network. This notion, in the context of (standard) threshold encryption, was formalized and achieved as *silent threshold encryption* (STE) scheme by [20]. However, none of the previously discussed BTE schemes support a silent setup; and the STE scheme in [20] incurs the same communication overhead as standard threshold encryption for decrypting a batch of messages, i.e., the partial decryption size grows linearly with B for a batch of B messages.

Thus, although significant progress has been made in using threshold encryption to ensure mempool privacy, incorporating various desirable features, none of the existing schemes are suitable for the large open networks characteristic of blockchain environments.

1.1. Our Contribution

In this work, we address the above challenge and build the first practical construction for batched threshold encryption, that does not rely on any (costly) interactive setup protocol.

- We construct an efficient CCA-secure *silent* batched threshold encryption scheme. Our construction requires no interaction among committee members during the setup phase, and no setup is needed per epoch. In contrast to most prior work on BTE [16],

[17], [19], we achieve the additional benefit that ciphertexts are not tied to any specific block.
- As part of our construction, and of independent interest, we introduce the *first* efficient additive homomorphic silent threshold encryption scheme (HSTE).
- On a more engineering level, we propose several *novel* optimizations that enable efficient encryption and decryption of multiple messages simultaneously in our HSTE. Concretely, by applying these optimizations, the resulting ciphertext is up to $300\times$ smaller and partial decryptions are up to $96\times$ smaller. The techniques underlying these optimizations may also be of independent interest.
- We provide formal security proofs for our construction, covering practical attack scenarios relevant to encrypted mempools.
- We implement the CPA secure version of our batched threshold encryption scheme in about 2500 lines of Rust and evaluate its performance in Section 8. Encryption takes ≈ 16.4 ms irrespective of other system parameters such as committee size n and batch size B. The ciphertexts are ≈ 1 KiB in size and only 38% larger than ciphertexts from [20] which does not support batched decryption. Partial decryption takes 0.55 ms at a batch size of 512 and reconstruction of all messages can be completed in 129.5 s in single threaded mode with a constant partial decryption size. We emphasize that the reconstruction step can be trivially optimized as in [18] and parallelized.

1.2. Technical Highlight

Due to space constraints, we briefly highlight the core ideas behind our construction without getting into too many technical details. From a bird's eye view, our goal is to combine the batch threshold decryption (BTE) [18] and silent threshold encryption (STE) [20], thereby achieving silent batch threshold encryption (SBTE). This comes with several technical hurdles, which we solve with various innovative ideas (that may be of independent interests).

Key Question. In a nutshell, the BTE [18], among various other components, relies on a homomorphic threshold encryption scheme (in particular, threshold ElGamal) to encrypt *source* group elements, which is actually the only part that requires DKG setup and, thus, the natural question is if we can replace the threshold ElGamal with STE [20] to get batch threshold encryption with silent setup. Unfortunately, this is infeasible since STE [20], as described, does not support any homomorphism.

Theoretical Innovation. We solve this problem by twisting both BTE and STE to make them compatible with each other. On the one hand, we observe that BTE can be made to work if we have a threshold encryption with homomorphism over *field elements* (as opposed to source group elements). On the other hand, we observe that STE [20] with some modification can support homomorphism over *target* group elements. At first glance, this might not seem very useful, as the homomorphisms still do not match — enabling homomorphic encryption of field elements using target group homomorphism typically requires encoding a field element $x \in \mathbb{F}$ as g_T^x in the target group, which renders decryption infeasible. We solve this problem by employing binary decomposition. Namely, to encrypt x, one decomposes it as $(x_1, \ldots, x_\lambda)$ and encrypts each bit x_i by encrypting $g_T^{x_i}$. This approach preserves the required homomorphic properties while allowing for efficient decryption.

Practical Optimization. Unfortunately, the solution as presented above, while feasible, is practically too inefficient. The key reason is that the binary decomposition incurs an overhead of $\lambda = 128$. To make our construction practical, we introduce several optimizations. The first simple idea is that, instead of binary decomposition, we could use a larger base (say, $x = (x_1, \ldots, x_\ell)$ for some smaller ℓ), striking a balance between decryption hardness and ciphertext overhead. Second, we observe that, since the encryptor employs STE to encrypt a *batch* of plaintexts $(x_1, \ldots, x_\ell)$. Instead of naïvely encrypting each plaintext individually and incurring an overhead of ℓ, can we leverage *batch encryption* to obtain *(amortized) "rate-1"*?[2] We positively answer this question by presenting further modifications to STE to enable batch STE ciphertexts with rate-1. This modification is quite technical and requires careful attention to detail; we refer the readers to Section 6.2. Finally, we introduce a *general* technique to improve efficiency whenever one needs to send a target group element (as part of the ciphertext). In particular, if one wants to send a target group element h_T, one may *multiplicatively share* it as source group elements h_1 and h_2 such that $e(h_1, h_2) = h_T$. This is feasible as long as one knows the source group element (from either source group) that has the same discrete log as h_T and saves communication as target group elements

2. To illustrate this point, think of the classical ElGamal *batch* encryption. Namely, to batch encrypt $x_1, x_2, \ldots, x_\ell$ under public keys $\mathsf{pk}_1, \ldots, \mathsf{pk}_\ell$, one could encrypt it as $g^r, \mathsf{pk}_1^r \cdot g^{x_1}, \ldots, \mathsf{pk}_\ell^r \cdot g^{x_\ell}$, which is "rate-1". That is, ℓ encryptions require sending only $\ell + 1$ group elements.

are much larger than $\mathbb{G}_1$ and $\mathbb{G}_2$ elements combined.

1.3. Related Work

Mempool Privacy: MEV poses a significant challenge in blockchain systems. A promising mitigation strategy involves achieving mempool privacy, where threshold encryption has emerged as the de facto approach.

In constructions based on traditional threshold encryption schemes [14], [15], every transaction needs to be decrypted individually, which introduces a heavy $O(nB)$ communication overhead for n committee members and B ciphertexts. Subsequent works [21], [22], [23] employ identity-based encryption (IBE) to improve efficiency. Users now encrypt their transactions to a future block identity, and all transactions encrypted to the same block get decrypted once. This reduces decryption communication to $O(n)$. However, such approach compromises pending transaction privacy. As detailed earlier, batched threshold encryption [16], [17], [18], [19], [24] resolves the limitation efficiently.

Other primitives, like time-lock puzzles [25], [26], indistinguishability obfuscation (iO) [27], or general-purpose witness encryption [28], [29], can also be used to achieve mempool privacy. However, limitations like heavy computational overhead or reliance on strong cryptographic assumptions make these schemes less preferred for blockchain applications. Designing practical constructions under these primitives remains an open research question.

Alternative MEV Mitigation Strategies: Beyond cryptographic solutions for mempool privacy, several alternative approaches address MEV. In Ethereum, Flashbots and MEV-boost are used to regulate MEV extraction through auctions [9], [10]. While reducing its negative effects, they do not eliminate the problem and introduce centralization risks. Solutions based on Trusted-Execution Environments (TEEs) are also explored to solve MEV [12], [30]. The secret key of a public key encryption scheme is stored in secure enclaves, and the TEE will decrypt and publish the blocks of transactions encrypted to the corresponding public key. They inherit hardware trust assumptions and remain vulnerable to side-channel attacks [31], [32], which creates risks and makes them impractical solutions. Another line of work approaches the MEV problem by achieving fair ordering [13], [33], [34]. They enforce consensus over all miners on the transaction ordering, thus limiting any reordering by a single miner. However, only weakened order fairness is achieved so far, which does not eliminate MEV entirely. MEV-resistant DEXs such as [35], [36] employ game theoretic mechanisms to prevent MEV. These works are mostly complex and specific to certain applications, and cannot be used as a general solution. Motivated by these limitations, we build a mempool encryption scheme with the desired properties that practically mitigates MEV.

Silent Setup for Threshold Cryptography: DKG remains a persistent bottleneck in deploying threshold cryptosystems despite decades of research [37], [38], imposing substantial communication and computation overhead.

Threshold signatures are an essential primitive for decentralized systems like Ethereum, and similar to threshold decryption, they commonly rely on DKG protocols for key setup. Some prior works have attempted to remove the need for DKG by using multisignature [39], [40], which can be made to support non-interactive t-out-of-n key aggregation by having the signature aggregator include all signers, and letting the verifier aggregate the public keys of the desired t signers. This incurs linear signature size and verification costs, which are expensive for blockchain deployment. An intuitive way to improve the overhead is to let the aggregator also compute the public key aggregation. A succinct non-interactive argument (SNARK) is required to prove the correctness of this aggregated public key. Since a simple plug-in of generic SNARKs [41], [42] introduces high aggregation overhead, recent works [43], [44], [45] design custom SNARK algorithms to achieve concretely efficient aggregation time.

Beyond threshold signatures, there has been a line of research on silent setup for other advanced encryption schemes, including distributed/flexible broadcast encryption [46], [47], [48], registration-based encryption [49], [50], [51], registered attribute-based encryption [52], [53] and registered functional encryption [54], [55]. Recently, Garg et al. [20] present the first efficient construction of threshold encryption with silent setup, which inspires this work. Alternatively, other works attempt to construct threshold encryption with silent setup using iO [29], [56].

Critically, no existing work achieves silent setup for batched threshold encryption, which we believe to be a promising primitive for scalable, privacy-preserving mempools. Our work bridges this gap by defining and constructing the first concretely practical Silent Batched Threshold Encryption (SBTE) scheme.

2. Preliminaries

Notation. We denote the security parameter with $\lambda \in \mathbb{N}$. We write $\mathsf{negl}(\lambda)$ for functions that are negligible in λ. For integers a, b with $a \leq b$, we denote the sets $[a, b] = \{a, a+1, \ldots, b-1, b\}$ and $[b] = \{1, 2, \ldots, b\}$. We write $y \leftarrow A(x; r)$ to denote the execution of algorithm A on input x with randomness r, which outputs y. If A is executed with randomness that is sampled uniformly at random from the appropriate space, we abbreviate as $y \leftarrow\!\!\$\ A(x)$. We use $\approx_c$ to denote computational indistinguishability of random variables. For a probabilistic algorithm A, we write $y \in A(x)$ to denote that $\Pr[A(x) = y] > 0$.

Polynomials in $\mathbb{F}_p$. We define $\mathbb{H}_n = \{\omega, \omega^2, \ldots, \omega^n\}$ to be the multiplicative subgroup generated by the n-th root of unity $\omega \in \mathbb{F}_p$ (i.e. $\omega^n = 1$) with $|\mathbb{H}_n| = n$. Let $Z(x) = \prod_{i \in n}(x - \omega^i) = x^n - 1$ be the vanishing polynomial on $\mathbb{H}_n$. Let $L_i(x)$ be the Lagrange coefficient of interpolation at position x for index i with respect to $\mathbb{H}_n$ such that $L_i(\omega^j) = 0$ for all $j \neq i$ and $L_i(\omega^i) = 1$.

Univariate Sumcheck. We make use of the univariate sum check protocol [57], [58] generalized to inner products [59].

Lemma 1 (Univariate Sumcheck). *Let $A(x) = \sum_{i=1}^{|\mathbb{H}|} a_i \cdot L_i(x)$, and $B(x) = \sum_{i=1}^{|\mathbb{H}|} b_i \cdot L_i(x)$. It holds that*

$$A(x)B(x) = \frac{\sum_i a_i b_i}{|\mathbb{H}|} + Q_x(x)x + Q_Z(x)Z(x),$$

where both Q_x and Q_Z are polynomials of degree $\leq |\mathbb{H}| - 2$ defined as

$$Q_x(x) = \sum_i a_i b_i \frac{L_i(x) - L_i(0)}{x}, \text{ and}$$

$$Q_Z(x) = \sum_i a_i b_i \frac{L_i^2(x) - L_i(x)}{Z(x)} + \sum_{i \neq j} a_i b_j \frac{L_i(x)L_j(x)}{Z(x)}.$$

Pairings. A bilinear pairing ensemble $\mathcal{G} = (\mathbb{G}_1, [1]_1, \mathbb{G}_2, [1]_2, \mathbb{G}_T, p, \circ)$ is an ensemble of cyclic groups $\mathbb{G}_1$ with generator $[1]_1$, $\mathbb{G}_2$ with generator $[1]_2$, and $\mathbb{G}_T$ of prime order p with a pairing operation $\circ\colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ that satisfies bilinearity and non-degeneracy. We usually write $\circ$ infix.

For some of our proofs, we will use the Generic Group Model (GGM) [60], [61] and its *master theorem* [62], [63, Theorem 1]. We recall relevant definitions and theorems in Appendix A.1.

Cryptographic Building Blocks. We rely on a *commitment scheme* and *non-interactive zero-knowledge proofs* (NIZKs) as cryptographic building blocks.

At a high level, a *commitment scheme* allows to commit to a message m, generating a commitment com and an opening r such that: (a) the commitment com does not leak information about the message m (hiding); and (b) it is infeasible to find an opening r' such that com opens to a *different* message $m' \neq m$.

In non-interactive zero-knowledge proofs (of knowledge), a prover non-interactively convinces a verifier that they know a witness ω to a statement χ such that the statement-witness pair (χ, ω) is in some relation R. Crucially, the proof π cannot leak any additional information about ω (zero-knowledge), and a malicious prover should not be able to convince the verifier without knowledge of ω (knowledge-soundness / extractability).

Key-Homomorphic PPRFs Finally, we use *key-homomorphic puncturable pseudorandom functions* (PRF), which extends a standard PRF with a *key-homomorphism* (i.e., $\mathsf{PRF}(k_1, i) + \mathsf{PRF}(k_2, i) = \mathsf{PRF}(k_1 + k_2, i)$), and *puncturability*. Puncturability allows the key k to be "punctured" at a point i, producing a key k^*, which allows the PRF to be evaluated on any index except i. In more detail, this implies that, given k^*, $\mathsf{PRF.Eval}(k, i)$ looks random to a computationally bounded adversary.

Definition 2 (Key-Homomorphic Puncturable PRFs). *A key-homomorphic puncturable PRF family with bounded domain $[B]$ is a tuple of PPT algorithms $\mathsf{PRF} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{Punct}, \mathsf{Eval}, \mathsf{PEval})$ such that*

- $\mathsf{pp} \leftarrow\!\!\$\ \mathsf{Setup}(1^\lambda, B)$*: Given the security parameter λ as well as the domain bound B, the* Setup *algorithm returns public parameters* pp *that are implicit input to all subsequent algorithms.*
- $k \leftarrow\!\!\$\ \mathsf{KGen}(1^\lambda)$*: Given the security parameter λ, the key generation algorithm returns a key k.*
- $k^* \leftarrow \mathsf{Punct}(k, i^*)$*: Given a key k, and an index $i^* \in [B]$, the puncturing algorithm outputs a punctured key k^*.*
- $y \leftarrow \mathsf{Eval}(k, i)$*: Given key k and index $i \in [B]$, the* Eval *algorithm returns the evaluation y.*
- $y \leftarrow \mathsf{PEval}(k^*, i^*, i)$*: Given a punctured key k^* for index $i^* \in [B]$ and an index $i \neq i^*$ with $i \in [B]$, the punctured evaluation algorithm returns the evaluation y.*

Sometimes we write $\mathsf{PRF}(k, i)$ as a shorthand for $\mathsf{PRF.Eval}(k, i)$. We require the PRF to be correct and key-homomorphic, i.e., for all $\lambda, B \in \mathbb{N}$, for all $k_1, k_2 \in \mathsf{KGen}(\lambda)$, and all $i, i^* \in [B]$ with $i \neq i^*$ it holds that

- $\mathsf{Eval}(k_1, i) + \mathsf{Eval}(k_2, i) = \mathsf{Eval}(k_1 + k_2, i)$, and
- $\mathsf{Eval}(k_1, i) = \mathsf{PEval}(\mathsf{Punct}(k_1, i^*), i^*, i)$.

Additionally, we require that the outputs of the PRF are *pseudorandom*, which we formally define in Appendix A.4.

In our constructions, we instantiate the PRF with a pairing-based construction from [64] where keys $k \in \mathbb{F}_p$ are field elements, punctured keys $k^* \in \mathbb{G}_1$ are source-group elements, and evaluations $y \in \mathbb{G}_T$ are target-group elements.

In the following, we denote as λ_k the size of the key in bits. For BLS-12-381, we have that $\lambda_k = 256$ for $\lambda = 128$.

We formally define the above building blocks within Appendix A.

3. Definitions

To model security, we elevate the definition of a batched threshold encryption scheme [18] to the silent setup setting [20].

Definition 3 (Silent Batched Threshold Encryption)**.** *A* Silent Batched Threshold Encryption *scheme (*SBTE*) consists of a tuple of PPT algorithms* $\mathsf{SBTE} = (\mathsf{Setup}, \mathsf{KGen}, \mathsf{KVfy}, \mathsf{Prep}, \mathsf{Enc}, \mathsf{Vfy}, \mathsf{BatchDec}, \mathsf{PDVfy}, \mathsf{Comb})$ *with the following syntax.*

- $\mathsf{pp} \leftarrow\!\$\ \mathsf{Setup}(1^\lambda, B_{\max}, N)$*: This algorithm initializes the scheme, receiving the security parameter* $\lambda \in \mathbb{N}$*, the maximum batch size* $B_{\max}$*, and the maximum number of members per committee* N*. It produces public parameters* pp*, which are an implicit input to all subsequent algorithms.*
- $(\mathsf{pk}, \mathsf{sk}, \pi^{\mathsf{KGen}}) \leftarrow\!\$\ \mathsf{KGen}(1^\lambda)$*: The key generation algorithm returns a key pair* $(\mathsf{pk}, \mathsf{sk})$ *as well as a proof* π^{KGen}*.*
- $1/0 \leftarrow \mathsf{KVfy}(\mathsf{pk}, \pi^{\mathsf{KGen}})$*: The key verification algorithm gets a public key* pk *as well as a proof* π^{KGen} *and outputs* 1 *to indicate the public key is valid and* 0 *otherwise.*
- $(\mathsf{ek}, \mathsf{ak}) \leftarrow \mathsf{Prep}(\mathcal{C}, \{\mathsf{pk}_i\}_{i\in\mathcal{C}})$*:* Prep *is a deterministic algorithm that receives a committee* $\mathcal{C} \subseteq [N]$ *of at most* N *parties and their public keys. It outputs a concise encryption key* ek *and aggregation key* ak*.*
- $(\mathsf{ct}, \pi^{\mathsf{ct}}) \leftarrow\!\$\ \mathsf{Enc}(\mathsf{ek}, m, i, t)$*: Given an encryption key* ek*, a message* m*, an index* i*, and a threshold* $1 \le t \le N$*, the encryption algorithm outputs the ciphertext* ct *for batch position* i*, along with a proof* π^{ct}*.*
- $1/0 \leftarrow \mathsf{Vfy}(\mathsf{ek}, \mathsf{ct}, \pi^{\mathsf{ct}})$*:* Vfy *is a deterministic algorithm that takes as input an encryption key* ek*, a ciphertext* ct*, and a proof* π^{ct}*. It outputs* 1 *if the proof is valid and* 0 *otherwise.*
- $(\mathsf{pd}, \pi^{\mathsf{pd}})/\perp \leftarrow \mathsf{BatchDec}(\mathsf{sk}, \{\mathsf{ct}_i\}_{i\in[B]})$*: Given a secret key* sk *and a batch of ciphertexts* $\{\mathsf{ct}_i\}_{i\in[B]}$ *where* $B \le B_{\max}$*, the decryption algorithm generates a partial decryption* pd *along with a proof* π^{pd}*, or returns* $\perp$*.*
- $1/0 \leftarrow \mathsf{PDVfy}(\mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}, \mathsf{pk}, \pi^{\mathsf{pd}})$*: Given a partial decryption* pd*, a batch of ciphertexts* $\{\mathsf{ct}_i\}_{i\in[B]}$*, a public key* pk*, and a proof* π^{pd}*, the partial decryption verification algorithm returns 1 to indicate that the partial decryption is valid and 0 otherwise.*
- $\{m_i/\perp\}_{i\in[B]} \leftarrow \mathsf{Comb}(\mathsf{ak}, \{\mathsf{pd}_j\}_{j\in S}, \{\mathsf{ct}_i\}_{i\in[B]})$*: The combining algorithm takes as input an aggregation key* ak*, a set of partial decryptions* $\{\mathsf{pd}_j\}_{j\in[S]}$*, and a batch of ciphertexts* $\{\mathsf{ct}_i\}_{i\in[B]}$*. It outputs the decrypted messages* $\{m_i\}_{i\in[B]}$ *or an error symbol* $\perp$*.*

We require that $B_{\max}$ *and* N *are polynomial in* λ*.*

Efficiency. *The size of the partial decryption output by* BatchDec *must be sublinear in* B*, i.e.,* $o(B)$*. Additionally, we require that each party executes* BatchDec *independently, that is,* BatchDec *is non-interactive and its runtime must be independent of* N*, i.e., it should run in time* $\mathsf{poly}(B, \lambda)$*.* Setup *must run in time* $\mathsf{poly}(N, B_{\max}, \lambda)$*,* Enc *in* $O(1)$*, and* Comb *in* $\mathsf{poly}(N, B_{\max}, \lambda)$*.*

Remark 4. *For simplicity, we present our construction for committees of size* $|\mathcal{C}| = N$*. However, we note that our constructions can easily be extended to support smaller committees* $|\mathcal{C}| < N$*.*

Remark 5. *We note that we explicitly define* Enc *to output a separate proof* π^{ct} *and introduce a dedicated verification algorithm* Vfy *for ciphertexts, rather than incorporating verification into* BatchDec*. This design reflects the requirement that ciphertexts must be publicly verifiable. Public verifiability is particularly important in the mempool application, where ciphertexts must be verifiable by block proposers.*

Following [18], we first introduce our definitions and constructions in the *coordinated* setting, where each ciphertext in a batch has a unique index i that is known at the time of encryption. Note that we can use the same sub-batching technique as [18] to remove coordination.

Definition 6 (Correctness)**.** *An* SBTE *scheme* Σ *is correct, if*

$$\forall i \in [\mathcal{C}]\colon \mathsf{KVfy}(\mathsf{pk}_i, \pi_i^{\mathsf{KGen}}) = 1$$
$$\forall j \in [B]\colon \mathsf{Vfy}(\mathsf{ek}, \mathsf{ct}, \pi^{\mathsf{ct}}) = 1$$
$$\forall i \in S\colon \mathsf{PDVfy}(\mathsf{pd}_i, \{\mathsf{ct}_j\}_{j\in[B]}, \mathsf{pk}, \pi^{\mathsf{pd}}) = 1$$
$$\mathsf{Comb}(\mathsf{ak}, \{\mathsf{pd}_i\}_{i\in S}, \{\mathsf{ct}_j\}_{j\in[B]}) = \{m_j\}_{j\in[B]},$$

for $\lambda, B, t, B_{\max}, N \in \mathbb{N}$, $B \leq B_{\max}$, $\mathcal{C} \subseteq [N]$, *and* $S \subseteq |\mathcal{C}|$ *with* $t \leq |\mathcal{C}|$, *and* $|S| \geq t$, *and for all messages* $\{m_j\}_{j\in[B]}$, *and all*

- $\mathsf{pp} \in \mathsf{Setup}(1^\lambda, B_{\max}, N)$
- $\{(\mathsf{pk}_i, \mathsf{sk}_i, \pi_i^{\mathsf{KGen}}) \in \mathsf{KGen}(1^\lambda)\}_{i\in\mathcal{C}}$
- $(\mathsf{ek}, \mathsf{ak}) \in \mathsf{Prep}(\mathcal{C}, \{\mathsf{pk}_i\}_{i\in\mathcal{C}}$
- $\{(\mathsf{ct}_j, \pi_j^{\mathsf{ct}} \in \mathsf{Enc}(\mathsf{ek}, m_j, j, t)\}_{j\in[B]}$
- $\{(\mathsf{pd}_i, \pi_i^{\mathsf{pd}}) \in \mathsf{BatchDec}(\mathsf{sk}_i, \{\mathsf{ct}_j\}_{j\in[B]})\}_{i\in S}$.

To model security, we adjust the definition from [18] of B-IND-CCA and rogue ciphertext security to the silent-setup setting.

Definition 7 (SB-IND-CCA-security)**.** *An* SBTE *scheme* Σ *is* SB-IND-CCA*-secure if for all PPT-adversaries* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ *there exists a negligible function* negl *such that*

$$\Pr[\mathsf{Game\text{-}SB\text{-}IND\text{-}CCA}^{\Sigma}_{\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-SB-IND-CCA *is defined in Figure 1.*

Rogue Ciphertext Security. [18] introduced the notion of Rogue Ciphertext Security to provide security against an adversary that introduces malformed ciphertexts into the batch of ciphertexts being decrypted such that correctness no longer holds for honestly generated ciphertexts.

Definition 8 (Rogue Ciphertext security)**.** *An* SBTE *scheme* Σ *is rogue ciphertext secure if for all PPT adversaries* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ *there exists a negligible function* negl *such that*

$$\Pr[\mathsf{Game\text{-}Rogue}^{\Sigma}_{\mathcal{A}}(1^\lambda) = 1] \leq \mathsf{negl}(\lambda),$$

where Game-Rogue *is defined in Figure 1.*

Remark 9. *Note that Silent Threshold Encryption* STE *is a special case of* SBTE *with* $B_{\max} = 1$. *Therefore, we do not give a separate definition for* STE. *In* STE, *we call the decryption algorithm* Dec *instead of* BatchDec.

For the sake of completion, we also recall the semantic security definition for silent setup encryption (without batching), following [20].

Definition 10 (S-IND-CPA-security)**.** *An STE scheme* Σ *is* S-IND-CPA*-secure if for all PPT-adversaries* $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ *there exists a negligible function* negl *such that*

$$\Pr[\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}^{\Sigma}_{\mathcal{A}}(1^\lambda) = 1] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where Game-S-IND-CPA *is defined in Figure 1.*

4. Additive Homomorphic STE

In this section, we present a variant of the construction from [20], modified to support additive homomorphism. This property is essential for applying the framework of [18] to construct a batched threshold encryption scheme, as we discuss in Section 5.

4.1. The Σ_{HSTE} Construction

Below, we describe a homomorphic version of the CPA-secure silent threshold encryption scheme due to [20]. We focus solely on the core algorithms in this description. Note that the presentation does not strictly adhere to the formal definition of STE.

Setup: Sample $\tau \leftarrow\!\$\ \mathbb{F}$ and $\gamma \leftarrow\!\$\ \mathbb{F}$ and publish $\mathsf{crs} = \left([\gamma]_2, [\tau^1]_1, \ldots, [\tau^n]_1, [\tau^1]_2, \ldots, [\tau^n]_2\right)$. Note that the tag γ is now included in the crs, allowing it to be reused across all ciphertexts.

KeyGen: The i-th party for $i \in [n]$ samples $\mathsf{sk}_i \leftarrow\!\$\ \mathbb{F}_p$ and publishes $\mathsf{pk}_i = ([\mathsf{sk}_i]_1, [\mathsf{sk}_i \cdot \tau]_1, \ldots, [\mathsf{sk}_i \cdot \tau^{n-1}]_1)$.

Preprocessing: Compute the encryption public key $\mathsf{ek} = [\mathsf{SK}(\tau)]_1 = [\sum_{i=1}^n \mathsf{sk}_i \cdot L_i(\tau)]_1$.

Before proceeding with the construction, we provide a brief overview of the ideas behind the encryption and decryption procedures. The encryption procedure is formulated as a witness encryption scheme for a certain relation, making decryption possible only with knowledge of the corresponding witness. We will now describe this particular relation; the details will become clear shortly. The linear relation is as follows: $\mathbf{A} \circ \mathbf{w} = \mathbf{b}$, where

$$\mathbf{A} = \begin{bmatrix} [\mathsf{SK}(\tau)]_1 & [1]_2 & [Z(\tau)]_2 & [\tau]_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & [\tau^2]_2 & [1]_2 & 0 & 0 & 0 \\ 0 & [\gamma]_2 & 0 & 0 & 0 & [1]_1 & 0 & 0 \\ [\tau^t]_1 & 0 & 0 & 0 & 0 & 0 & [1]_2 & 0 \\ [1]_1 & 0 & 0 & 0 & 0 & 0 & 0 & [\tau]_1 \end{bmatrix}$$

$$\mathbf{w} = \begin{bmatrix} [B(\tau)]_2 \\ [-\mathsf{aSK}]_1 \\ [-Q_Z(\tau)]_1 \\ [-Q_x(\tau)]_1 \\ [\hat{Q}_x(\tau)]_1 \\ [\sigma^*]_2 \\ [-\hat{B}(\tau)]_1 \\ [-Q_0(\tau)]_2 \end{bmatrix}, \text{ and } \mathbf{b} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ [1]_T \end{bmatrix}. \tag{1}$$

This relation encodes five linear constraints that together ensure at least t parties have provided valid decryption shares. Each party contributes a partial decryption, which can be interpreted as a BLS signature on the tag $[\gamma]_2$ using its secret key.

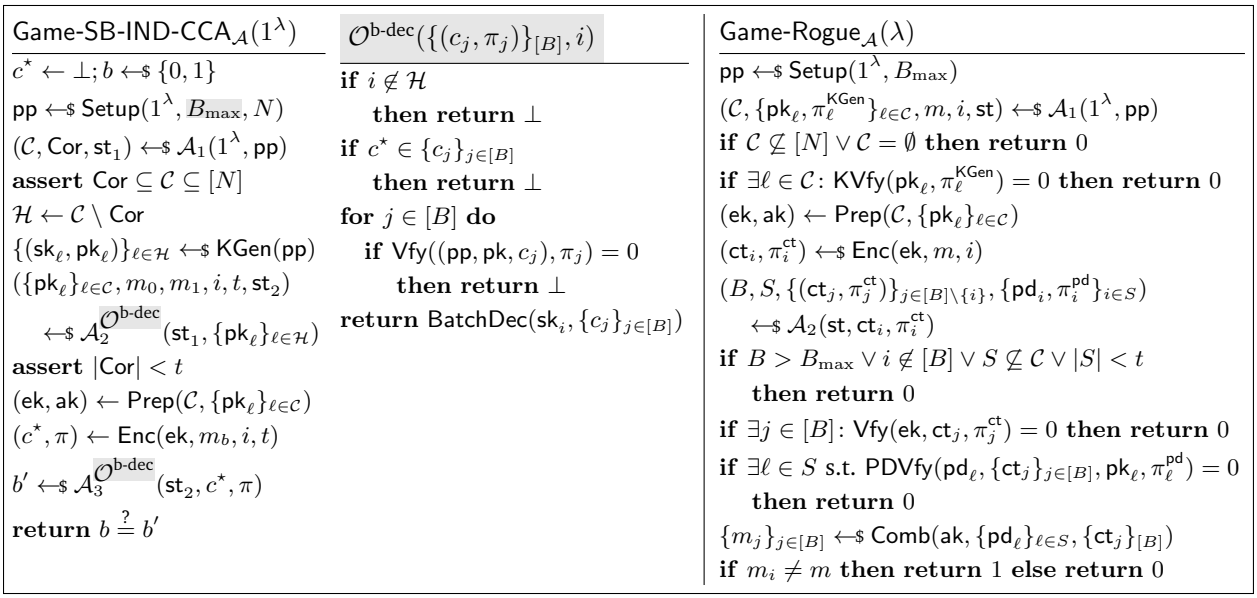

Figure 1. SB-IND-CCA and rogue ciphertext security game of SBTE. We also define the semantic security game for silent setup encryption [20] Game-S-IND-CPA as the left game, but without providing access to $\mathcal{O}^{\text{b-dec}}$ to the adversary, and limiting $B_{\max} = 1$ (highlighted in gray).

Decryption then requires a valid aggregated signature σ^* on $[\gamma]_2$ derived from t valid shares.

For every subset $S \subseteq [n]$ with $|S| \geq t$, there exists a unique aggregated public key $\mathsf{aPK}_S = [-\mathsf{aSK}]_1$, derived from the corresponding individual public keys. The aggregated signature σ^* must then be verified against aPK_S. This verification is captured by the third linear constraint:

$$[-\mathsf{aSK}]_1 \circ [\gamma]_2 = [1]_1 \circ [\sigma^*]_2.$$

To ensure that aPK_S is a valid aggregated key corresponding to a qualified set S with $|S| \geq t$, we define a polynomial B such that $B(\omega^i) = 0$ for all $i \notin S$. The correctness of aPK_S is then enforced by the first constraint, derived from the sumcheck lemma (Lemma 1):

$$\mathsf{SK}(\tau)_1 \circ [B(\tau)]_2 = \mathsf{aPK}_S \circ [1]_2 + [Q_x(\tau)]_1 \circ [\tau]_2 + [Q_Z(\tau)]_1 \circ [Z(\tau)]_2$$

We enforce three additional constraints:

- B must represent an authorized subset, i.e., it is non-zero for at least t positions. This is enforced by the fourth constraint, which verifies that $deg(B) \leq n - t$ by checking that $[\tau^t] \circ [B(\tau)]_2 = [-\hat{B}(\tau)]_1 \circ [1]_2$.
- The degree of Q_x must be at most $|\mathbb{H}| - 2$, per the requirements of the sumcheck lemma. This is enforced by the second constraint which translates to a degree check on Q.
- $B(0) = 1$ to rule out the trivial (zero) polynomial. This is enforced by the fifth constraint which translates to KZG [65] verification: $[1]_1 \circ [B(\tau)]_2 = [\tau] \circ [Q_0(\tau)]_2$.

These constraints collectively verify that $|S| \geq t$. We elaborate further in the decryption phase.

Encrypt: To encrypt a message $m \in \mathbb{G}_T$ to public keys $(\mathsf{pk}_1, \ldots, \mathsf{pk}_n)$, sample $\mathbf{s} = (s_1, \ldots, s_5) \leftarrow\$ \mathbb{F}^5$. The ciphertext is then computed as: $\mathsf{ct} = \big(\mathsf{ct}_1 = \mathbf{s}^\top \cdot \mathbf{A}, \mathsf{ct}_2 = \mathbf{s}^\top \cdot \mathbf{b} + m\big)$.

Partial Decryption: The i-th party computes $\sigma_i = \mathsf{sk}_i \cdot [\gamma]_2$, and publishes the partial decryption $\mathsf{pd}_i = \mathsf{ct}_{1,6} \circ \sigma_i$. Note that the element $\mathsf{ct}_{1,6}$ corresponds to $[\mathbf{s}_3]_1$.

Combine: Given t partial decryptions, let $S \subseteq [n]$ be the set of parties that published valid partial decryptions. The combiner executes the following:

1) Interpolate a polynomial $B(X)$ such that $B(0) = 1$, and $\{B(\omega^i) = 0\}_{i\in[n]\setminus S}$.
2) Compute the aggregated public key corresponding to the set S: $[\mathsf{aSK}]_1 = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i) \cdot [sk_i]_1\right)$
3) Define the polynomials $Q_x(X)$, and $Q_Z(X)$ that are required in the sumcheck lemma (Lemma 1). That is: $\mathsf{SK}(X) \cdot B(X) = \mathsf{aSK} + Q_x(X)X + Q_Z(X)Z(X)$. Evaluate $[B(\tau)]_2$, $[-Q_x(\tau)]_1$, and $[-Q_Z(\tau)]$.

4) Aggregate the partial decryptions: $\mathsf{pd}^* = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i)\cdot \mathsf{pd}_i\right)$.
5) Set $\hat{Q}_x(X) = Q_x(X)\cdot X^2$ and evaluate $[\hat{Q}_x(\tau)]_2$. This is used to prove that Q_x is of degree at most $n-2$.
6) Set $Q_0(X) = (B(X)-1)/X$ and evaluate $[Q_0(\tau)]_2$. This is a KZG proof that $B(0)=1$.
7) Set $\hat{B}(X) = B(X)\cdot X^t$ and evaluate $[\hat{B}(\tau)]_1$. This is used to prove that B is of degree at most $n-t$.

Then assemble the witness $\mathbf{w}$ as in Eq. (1), except that we ignore the sixth element (by putting $\perp$ there) and instead we compute $[m]_T \leftarrow \mathsf{ct}_2 - (\mathsf{ct}_1 \circ \mathbf{w} + \mathsf{pd}^*)$.

Correctness and Additive Homomorphism. Follows by inspection for all ciphertexts that use the same tag $[\gamma]_2$.

Semantic Security. We provide a full security proof, accounting for our modifications over [20] and further optimizations introduced in Section 6, in Appendix B.1.

Our Modifications to [20]. First, in contrast to [20], where each ciphertext uses a fresh random γ, we use the same γ across all ciphertexts. Second, we modify the partial decryption. In [20], the partial decryption is simply σ_i. However, when a single γ is shared across all ciphertexts, such a partial decryption is no longer bound to any specific ciphertext, causing all ciphertexts to be decrypted simultaneously. To avoid this, in pd_i, we bind each partial decryption to a specific ciphertext using $[\mathbf{s}_3]_1$, which is unique to each ciphertext. This binding is crucial for proving the CCA security of the silent batch threshold encryption scheme presented in Section 7.

5. Silent Batched Threshold Encryption from Σ_{HSTE}

In this section, we demonstrate how to apply the framework from [18] using an additive homomorphic STE. The construction outlined here focuses on the core ideas underlying the design of silent batched threshold encryption. At this stage, we do *not* address potential attacks that compromise CCA security (Definition 7) or rogue ciphertext security (Definition 8); these considerations are addressed in Section 7.1 as part of our final construction.

5.1. Recap of the BEAT-MEV Construction

We begin by describing BEAT-MEV, the batched threshold encryption scheme from [18], which leverages key-homomorphic puncturable PRFs. Let PKE denote an additively homomorphic threshold encryption scheme, and let PRF be a key-homomorphic puncturable PRF. The scheme is as follows:

- Setup: The committee runs PKE.Setup in order to generate a distributed public key pk as well as private secret shares $\mathsf{sk}_1,\ldots,\mathsf{sk}_n$. Each party in the committee receives a private secret share.
- Enc: Given a message m and a public key pk, the i-th client who is assigned the index $i\in[B]$, samples a PRF key k and punctures it at index i as $k^* \leftarrow \mathsf{Punct}(k,i)$. The client then computes the ciphertext $\mathsf{ct}_i = (k^*, \beta = m + \mathsf{PRF}(k,i), v = \mathsf{PKE.Enc}(\mathsf{pk},k))$.
- BatchDec: Given a batch of ciphertexts $\{\mathsf{ct}_1,\ldots,\mathsf{ct}_{|B|}\}$, each party P_j computes $\mathsf{pd}_j \leftarrow \mathsf{PKE.Dec}(\mathsf{sk}_j, \sum_j \mathsf{ct}_j.v)$.
- Comb: Given t partial decryptions recover $k = \sum_i k_i$ from $\mathsf{pd}_1,\ldots,\mathsf{pd}_t$, where k_i is the PRF key in the i-th ciphertext ct_i. Using k, we can compute $\mathsf{PRF}(k_i,i)$ for all $i\in[B]$ as follows:

  $$\mathsf{PRF}(k_i,i) = \mathsf{PRF}(k,i) - \sum\nolimits_{j\neq i} \mathsf{PEval}(\mathsf{ct}_j.k^*, i).$$

  Given $\mathsf{PRF}(k_i,i)$, we can recover $m_i \leftarrow \mathsf{ct}_i.\beta - \mathsf{PRF}(k_i,i)$.

Note that in BatchDec, the parties aggregate all ciphertexts in the batch and reveal partial decryptions only for the aggregated ciphertext. To enable this step, the underlying scheme PKE must be additively homomorphic. The construction in [18] leverages threshold ElGamal in the exponent as the underlying PKE. However, this PKE does not support plaintext recovery when the message is large, as is the case with the key k. In [18], they demonstrate that, instead of recovering the plaintext, obtaining g^k is sufficient to evaluate the PRF.

We next elaborate on how to remove the interactive setup using the homomorphic STE scheme as described in Section 4. This requires a bit-decomposition of the PRF keys.

5.2. Baseline SBTE Construction

Next, we describe a baseline SBTE construction that combines our Σ_{HSTE} with BEAT-MEV [18], followed by a discussion of its limitations.

Let PRF be a key-homomorphic puncturable PRF. The scheme is as follows:

- Setup: Execute Σ_{HSTE}.Setup and PRF.Setup. The output of the setup is a crs.

- KGen: Each party intending to participate in decryption locally runs $\Sigma_{\mathsf{HSTE}}.\mathsf{KGen}$ using the crs, and publishes its public key. Let $(\mathsf{pk}_1, \ldots, \mathsf{pk}_n)$ denote the set of public keys.
- Enc: Given a message $m_i \in \mathbb{G}_T$, the crs, the public keys, and an index $i \in [B]$:
  - Sample a PRF key k_i and puncture it at index i as $k_i^* \leftarrow \mathsf{Punct}(k, i)$. Compute $\beta_i = m_i + \mathsf{PRF}(k, i)$.
  - Decompose k_i into λ_k bits, and encrypt each bit using Σ_{HSTE} to the committee, yielding $(v_{i,1}, \ldots, v_{i,\lambda_k})$, where each $v_{i,j} = \Sigma_{\mathsf{HSTE}}.\mathsf{Enc}(k_{i,j})$ encrypts the j-th bit of k_i under the public keys $\{\mathsf{pk}_1, \ldots, \mathsf{pk}_n\}$.
  - Output $\mathsf{ct}_i = (k_i^*, \beta_i, v_{i,1}, \ldots, v_{i,\lambda_k})$.
- BatchDec: Given a batch of ciphertexts $\{\mathsf{ct}_1, \ldots, \mathsf{ct}_B\}$, each party $u \in [n]$ in the committee performs:
  - Homomorphic aggregation of the encrypted bits $\{v_j \leftarrow \sum_i \mathsf{ct}_i.v_{i,j}\}_{j \in [\lambda_k]}$.
  - Computation and publication of context-bound partial decryptions: $\{\mathsf{pd}_{u,j} = \Sigma_{\mathsf{HSTE}}.\mathsf{Dec}(\mathsf{sk}_u, v_j)\}_{j \in [\lambda_k]}$. These partial decryptions are bound to the specific batch context, ensuring they cannot be reused to decrypt ciphertexts encrypted under the same tag.
- Comb: Let $k = \sum_{i=1}^{B} k_i$ denote the aggregate PRF key. Given partial decryptions from a set $S \subseteq [n]$ such that $|S| \geq t$, the j-th bit of k, denoted k^j, can be recovered by aggregating the corresponding set of partial decryptions[3] $[k^j]_T = \Sigma_{\mathsf{HSTE}}.\mathsf{Comb}(\{\mathsf{pd}_{u,j}\}_{u \in S}, v_j)$. Then, using k, we can recover $\{m_i\}_{i \in [B]}$ using a similar strategy as in BEAT-MEV [18].

Limitations of the Construction. While the above construction is *technically* a batched-silent threshold encryption scheme, its concrete efficiency falls significantly short of the requirements for practical encrypted mempools.

First, the bit decomposition of the PRF key introduces an λ_k-fold blowup in the ciphertext size. Specifically, each Σ_{HSTE} ciphertext includes $3\mathbb{G}_1$ elements, $5\mathbb{G}_2$ elements, and $1\mathbb{G}_T$ element, totaling approximately 1200 bytes at the 128-bit security level using the BLS-12-381 curve. Since the PRF key is $\lambda_k = 256$ bits, this results in a ciphertext of over 300 KiB + $|m|$. On blockchains, transactions are typically smaller than 1 KiB, resulting in an overhead of around 300 times.

3. Note that $k^j \in [0, \lceil \log B \rceil - 1]$. Thus, the discrete logarithm can be efficiently recovered through brute force.

Moreover, each party in the committee publishes 256 target group elements as their partial decryption amounting to ≈ 147 KiB!

To put these overheads in context, consider a naive scheme for encrypted mempools. Each committee member simply samples an El Gamal key pair and publishes their public key. To encrypt their transaction clients first secret share their message and encrypt the i-th share under the i-th committee members public key. Indeed, this scheme does not have an interactive setup, but the ciphertext size grows with $O(n)$, and the partial decryptions of each party are $O(B)$ in length. Concretely, each ciphertext is 64 bytes for a 32 byte message and each partial decryption is 32 bytes. For typical parameters of $n = 512$ and $B = 1024$, the ciphertext size is 32 KiB (order of magnitude smaller!) and the partial decryption size from each party is 32 KiB.

Although the baseline construction is theoretically interesting, as it establishes a feasibility result of a silent batched threshold encryption, the performance overhead renders it impractical to solve the problem of building encrypted mempools. In Section 6, we present a number of optimizations aimed at making the construction suitable for real-world deployment.

6. Making Σ_{HSTE} Practical

In this section we present a series of optimizations that are applied to Σ_{HSTE} construction to make it feasible for practical use. When the resulting construction is instantiated within the framework of [18], it yields a practically efficient silent batched threshold encryption construction.

The optimizations are summarized as follows:

- Chunking: Rather than performing bit-decomposition, we partition the key into chunks of size ℓ. This approach introduces the challenge of ensuring decryption remains efficient, both computationally and in terms of memory usage.
- Reusing randomness across ciphertexts: This is crucial to reducing the ciphertext size. In most cryptographic schemes, reusing randomness breaks semantic security. Therefore, our goal is to design a method that allows randomness reuse without compromising the security of the scheme.
- Avoiding target group elements:
  - To compress the ciphertext, we "multiplicatively" share the $\mathbb{G}_T$ element in the ciphertext across $\mathbb{G}_1$ and $\mathbb{G}_2$.

– A novel method for computing partial decryptions that are tied to individual ciphertexts, unlike the method employed in [20]. Notably, this allows us to move partial decryptions to $\mathbb{G}_1$.

We now discuss these optimizations in more detail.

6.1. Chunking

Instead of encrypting every bit of k separately, resulting in λ ciphertexts, we decompose k into ℓ chunks $(k_1, \ldots, k_\ell)$ of size $z = \lambda_k/\ell$ bits such that $k = \sum_{j\in[\ell]} k_j \cdot 2^{(j-1)z}$. This allows us to reduce the number of Σ_{HSTE} ciphertexts from λ_k to ℓ, at the cost of now extracting the discrete logarithm from decrypted $(K_1, \ldots, K_\ell) \in \mathbb{G}_T^\ell$. For a reasonable choice of ℓ, this can be done efficiently using Pollard's rho method [66], [67] or baby-steps giant-steps [68] due to the restriction that $\mathsf{DLog}(K_j) \in [0, 2^{z+\log(B)} - 1]$. In our evaluation, we choose $\ell = 8$ chunks, leading to a chunk size of 32 bits. Larger ℓ leads to smaller ciphertext size, but more expensive dlog extraction during decryption.

6.2. Reusing Randomness across Chunks

When applying chunking batched-silent threshold encryption, we encrypt ℓ chunks of the PRF key using *independent* randomness to the same tag γ. This blows up the ciphertext and the partial decryption size by ℓ. To save on ciphertext cost, we propose *sharing* the randomness across the ciphertexts by *extending* the linear relation[4]. For every chunk $j \in [2, \ell]$, in the setup we sample $r_j \leftarrow\$ \mathbb{F}_p$. For ease of notation, we set $r_1 \coloneqq 1$ We define $\mathbf{w}_j$ and $\mathbf{b}_j$ as follows

$$\mathbf{w}_j = r_j \cdot \mathbf{w}, \text{ and } \mathbf{b}_j = r_j \cdot \mathbf{b}. \tag{2}$$

The extended relation for silent threshold encryption can be represented as: $\mathbf{A} \circ \mathbf{w}_j = \mathbf{b}_j$, for every $j \in [\ell]$.

In the witness encryption, each column $\mathbf{b}_j$ for $j \in [\ell]$ will now result in a separate *mask* which can be used to encrypt the j-th message m_j. Concretely, we sample randomness $\mathbf{s} = [s_1, s_2, s_3, s_4, s_5] \leftarrow\$ \mathbb{F}_p^5$ and set the ciphertext to be:

$$\begin{aligned}
\mathsf{ct}_1 &= s_1 \cdot [\mathsf{SK}(\tau)]_1 + s_4 \cdot [\tau^t]_1 + s_5 \cdot [1]_1,\\
\mathsf{ct}_2 &= s_1 \cdot [1]_2 + s_3 \cdot [\gamma]_2, \quad \mathsf{ct}_3 = s_1 \cdot [Z(\tau)]_2,\\
\mathsf{ct}_4 &= s_1 \cdot [\tau]_2 + s_2 \cdot [\tau^2]_2, \quad \mathsf{ct}_5 = s_2 \cdot [1]_2,\\
\mathsf{ct}_6 &= s_3 \cdot [1]_1, \quad \mathsf{ct}_7 = s_4 \cdot [1]_2, \quad \mathsf{ct}_8 = s_5 \cdot [\tau]_1,\\
\{\mathsf{ct}_{9,j} &= s_5 \cdot [r_j]_T + m_j \cdot [1]_T\}_{j\in[\ell]}.
\end{aligned}$$

The ciphertext size is now $5 \cdot \mathbb{G}_2 + 3 \cdot \mathbb{G}_1 + \ell \cdot \mathbb{G}_T$ to encrypt ℓ chunks, which saves $(\ell - 1)(5\mathbb{G}_2 + 3\mathbb{G}_1)$ elements, which amounts to around 4.3 KiB in savings for $\ell = 8$.

The decryption proceeds as in Σ_{HSTE}. However, we now run ℓ decryption instances: the i-th instance uses the column $\mathbf{b}_i$ and generates the witness $\mathbf{w}_i$.

Semantic Security. We provide some intuition for the security of this scheme. In the GGM, by treating the randomness chosen during encryption, the trapdoor τ, as well as $\{r_i\}_{i\in[2,\ell]}$ as formal variables, all terms in the relation can be viewed as formal polynomials in these variables. Then, we argue that given the view of the adversary, if the column $\mathbf{b}_j$ is *independent*of the columns of $\mathbf{A}$ and the remaining columns $\mathbf{b}_v$ for every $v \in [\ell] \setminus \{j\}$, then this implies that the mask corresponding to the j-th column is indistinguishable from a uniformly random group element, which in turn guarantees semantic security of the underlying message.

It now remains to argue that in the silent threshold encryption scheme, this condition does hold for every column $\mathbf{b}_j$ for $j \in [\ell]$.

It was already shown in [20, Lemma 2] that $\mathbf{b}$ is indeed independent of the columns of $\mathbf{A}$ for and other terms in the adversary's view in the baseline construction if it does not receive at least one additional partial decryption (signature on $[\gamma]_2$). Intuitively, we can extrapolate because, given the adversary's view, any given $[r_j s_5]_T$ is independent of all $[r_v s_5]_T$ for $v \neq j$.

6.3. Avoiding Target Group Elements

We describe how to avoid using target group elements ($\mathbb{G}_T$) in both the ciphertext and the partial decryptions.

In the Ciphertext. Instead of including $\mathbb{G}_T$ elements $\{[r_i s_5]_T + [m_i]_T\}_{i\in[\ell]}$ in the ciphertext, we multiplicatively secret-share them across $\mathbb{G}_1$ and $\mathbb{G}_2$. Concretely, we replace each $\mathbb{G}_T$ element with one element in $\mathbb{G}_1$ and one in $\mathbb{G}_2$, and send: $v \cdot (s_5[r_i]_1 + [m_i]_1)$ and $v^{-1} \cdot [1]_2$, where $v \leftarrow\$ \mathbb{F}_p$ is

4. [69] used a similar strategy but for the El Gamal and Cramer-Shoup encryption schemes.

chosen at random at the time of encryption. This reduces the size of the ciphertext by $\ell \cdot (576-(48+96))$ bytes. For $\ell = 8$, this amounts to 3.4 KiB in savings.

To further reduce the ciphertext size, we sample $v_i \leftarrow\$ \mathbb{F}_p$ for every $i \in [\ell]$ during setup and include the shares $[r_i v_i]_1$, $[v_i^{-1}]_2$ as part of the crs. These values are then reused across all ciphertexts. As a result, each ciphertext only needs to include the $\mathbb{G}_1$ element, while the corresponding $\mathbb{G}_2$ can be derived from the crs. At the time of encryption, we then set $\mathsf{ct}_{9,i} = (s_5 + m) \cdot [r_i v_i]_1$. This reduces the size of the ciphertext by an additional $\ell \cdot 96$ bytes, which amounts to 0.75 KiB for $\ell = 8$. Note that this works only if the message m is a field element. As a tradeoff, recovering $\{m_i\}_{i\in[\ell]}$ from $\{[m_i r_i]_T\}_{i\in[\ell]}$ requires running ℓ independent instances of the underlying discrete log solver, since each instance corresponds to a different base.

Putting It All Together. We now have a ciphertext that is

$$\underbrace{5 \cdot \mathbb{G}_2 + 3\mathbb{G}_1 + 8 \cdot \mathbb{G}_1}_{\mathsf{STE}} + \underbrace{\mathbb{G}_1}_{\mathsf{k}^*} + |m| = 1056 \text{ bytes} + |m|$$

on the BLS12-381 curve for $\ell = 8$.

In the Partial Decryption. Interestingly, we observe that the partial decryption share can be modified to be $[\mathsf{s}_3\mathsf{sk}]_1$ instead of $[\mathsf{s}_3]_1 \circ [\mathsf{sk}\gamma]_2$. This modification does not affect the decryption process, as the term $[\mathsf{s}_3\gamma\mathsf{sk}r_i]_T$ can still be computed via the pairing $[\mathsf{s}_3\mathsf{sk}]_1 \circ [\gamma r_i]_2$. Importantly, this change does not compromise semantic security, since the ciphertext remains unaltered. As a result, the size of the partial decryption is now one $\mathbb{G}_1$ element, which is of size only 48 bytes. Crucially, the size of the partial decryption remains constant regardless of the parameter ℓ. Without this optimization, the size of the partial decryption is $\ell \cdot \mathbb{G}_T$, corresponding to 4.5 KiB when $\ell = 8$. Our optimization thus achieves a $96\times$ size reduction per partial decryption.

We will later show that this modification preserves CCA security, a necessary property for our silent batched threshold encryption scheme.

6.4. The Optimized HSTE Construction

We next describe the optimized construction Σ_{MSTE} which incorporates the optimizations discussed above.

Setup(1^λ): Sample $\tau, \gamma \leftarrow\$ \mathbb{F}_p$, and $r_u, v_u \leftarrow\$ \mathbb{F}_p$ for every $u \in [\ell]$, set $r_1 = 1$, and publish $\mathsf{crs} = \left\{[r_u]_{1,2}, [r_u\gamma]_2, [r_u\tau]_{1,2}, \ldots, [r_u\tau^n]_{1,2}, [r_u v_u]_1, [v_u^{-1}]_2\right\}_{u\in[\ell]}$

KeyGen(1^λ): Sample $\mathsf{sk} \leftarrow\$ \mathbb{F}_p$ and publish $\mathsf{pk} = \{[r_u\mathsf{sk} \cdot \tau^j]_1\}_{u\in[0,\ell],j\in[0,n-1]}$. We denote $\mathsf{pk}_{u,j} := [r_u\mathsf{sk}\tau^j]_1$ for $u \in [\ell]$ and $j \in [0, n-1]$.

Preprocess: Compute the encryption public key $[\mathsf{SK}(\tau)]_1 = [\sum_{i=1}^n \mathsf{sk}_i \cdot L_i(\tau)]_1$.

Encrypt: To encrypt a message $(m_1, \ldots, m_\ell) \in \mathbb{F}^\ell$ to public keys $(\mathsf{pk}_1, \ldots, \mathsf{pk}_n)$, sample $\mathbf{s} \leftarrow\$ \mathbb{F}^5$. The ciphertext is then computed as: $\mathsf{ct} = \left(\mathsf{ct}_1 = \mathbf{s}^\top \cdot \mathbf{A}, \{\mathsf{ct}_{2,i} = (s_5 + m_i)[r_i v_i]_1\}_{i\in[\ell]}\right)$.

Dec$(\mathsf{sk}_i, \mathsf{ct})$: Output $\mathsf{pd}_i \leftarrow \mathsf{sk}_i \cdot \mathsf{ct}_{1,6}\big[= [\mathsf{sk}_i s_3]_1\big]$.

Combine$(\mathsf{crs}, \mathsf{ak}, \mathsf{ct}, \{\sigma_{i,j}\}_{i\in[n]})$: Given t partial decryptions, let $S \subseteq [n]$ be the set of parties that published partial decryptions. The combiner runs a similar algorirhm as in Σ_{HSTE}. In particular, it defines the polynomials $B, \hat{B}(X), Q, Q_Z, Q_x, \hat{Q}_x, Q_0(X)$ as in Σ_{HSTE}. Then, the witness $\mathbf{w}_j$ for every $j \in [\ell]$ is similar to the wintess in Σ_{HSTE}, but with respect to the j-th pair of generators $([r_j]_1, [r_j]_2)$. That is, the witness $\mathbf{w}_j$ is as in Eq. (2) (omitting the sixth element), where $[r_j\mathsf{aSK}]_1 = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i) \cdot [r_j sk_i]_1\right)$ for every $j \in [\ell]$.

It is straightforward to verify that each $\mathbf{w}_j$ is computable from the crs and the public keys.

Next, the combiner computes, for every $j \in [\ell]$: $\mathsf{pd}_j^* = \frac{1}{n}\left(\sum_{i\in S} B(\omega^i) \cdot \mathsf{pd}_i \circ [r_j\gamma]\right)$. Finally, given $\mathbf{w}_j$ and pd_j^*, compute: $[m_j r_j]_T = \mathsf{ct}_{2,j} \circ [v_j^{-1}]_2 - (\mathsf{ct}_1 \cdot \mathbf{w}_j + \mathsf{pd}_j^*)$.

Note that for a message $(m_1, \ldots, m_\ell) \in \mathbb{F}^\ell$, the decryption recovers $([m_1 r_1]_T, \ldots, [m_\ell r_\ell]_T) \in \mathbb{G}_T^\ell$. This is sufficient for our use case in constructing silent batched threshold encryption, as we later extract each m_i by solving the discrete logarithm with respect to the generator $[r_i]_T$.

Lemma 11 (Semantic Security.)**.** *The modified STE construction Σ_{MSTE} is semantically secure (Definition 18).*

A formal security proof of Lemma 11 is presented in Appendix B.1.

7. The Final Construction

In this section, we introduce the full-fledged silent batched threshold encryption. It builds upon the baseline construction from Section 5, replacing Σ_{HSTE} by Σ_{MSTE} as the underlying STE. In addition, we enhance the construction with non-interactive zero-knowledge proofs to achieve CCA and rogue ciphertext security. A full construction overview is depicted in Figs. 2 and 3.

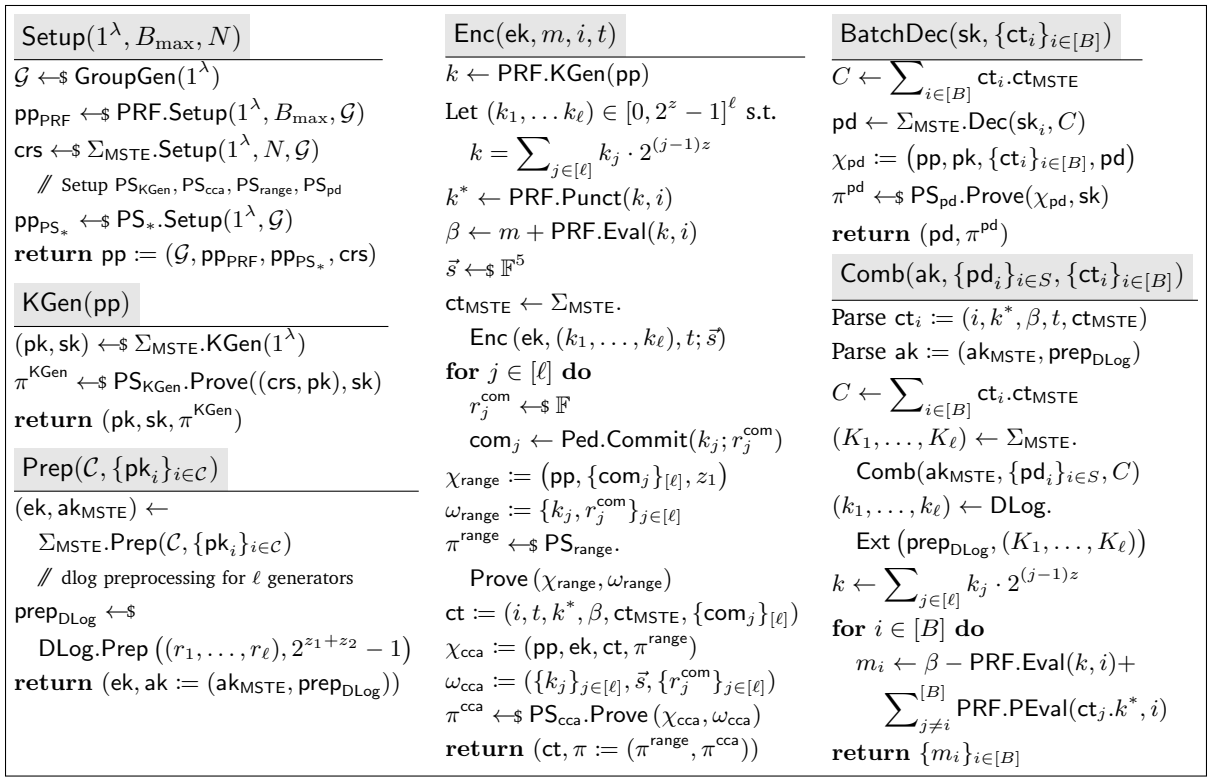

Figure 2. Construction of the SBTE scheme Σ_{SB} with ℓ chunks. We denote $z_1 = \lceil \lambda_k/\ell \rceil$ and $z_2 = \lceil \log(B_{\max}) \rceil$.

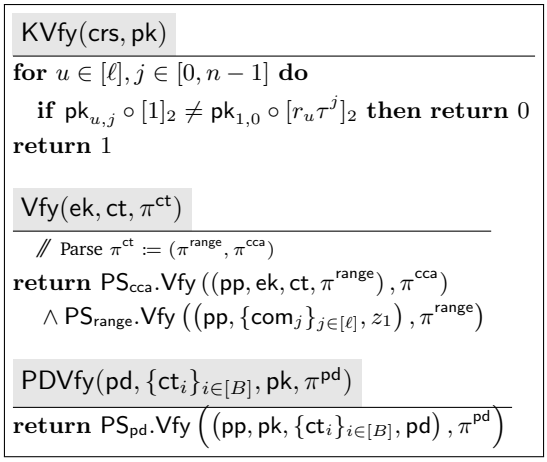

Figure 3. Verification algorithms of Σ_{SB}

As in the baseline construction, in encryption, the client samples a PRF key $k^{(i)}$. Instead of bit-decompostion, we now decompose the key into $\ell = 8$ chunks $(k_1^{(i)}, \ldots k_1^{(\ell)})$ and encrypt each sub-key using Σ_{MSTE}. In addition, the client attaches a Pedersen commitment to each $k_j^{(i)}$, a range proof π^{range} that all $k_j^{(i)} \in [0, 2^z - 1]$, as well as a proof of knowledge π^{cca} of the randomness involved. Both proofs are necessary to prevent rogue ciphertext attacks, and the latter allows us to achieve CCA security.

In partial decryption for a batch of ciphertexts, each committee member first verifies the corresponding proofs, and then generates the partial decryption as in Σ_{MSTE}. Each partial decryption is accompanied by a publicly verifiable proof π^{pd} to ensure correct decryption.

Given partial decryptions $\{\mathsf{pd}_i\}_{i\in S}$ from a set S of at least t committee members, the combiner first verifies the proofs associated with each partial decryption. It then aggregates the partial decryptions as in Σ_{MSTE}, yielding $(K_1, \ldots, K_\ell) \in \mathbb{G}_T^\ell$, where $K_m = \sum_{i\in[B]} k_m^{(i)} \cdot [r_m]_T$.

We apply the DLog extraction ℓ times to recover $(k_1, \ldots, k_\ell) \in \mathbb{F}_p^\ell$, reconstruct the key as $k = \sum_{j\in[\ell]} k_j \cdot 2^{(j-1)z}$. Observe that $k = \sum_{i\in[B]} k^{(i)}$, thus decryption proceeds as in BEAT-MEV.

7.1. NIZK Proofs for Rogue Ciphertext Prevention and CCA Security

In our construction, we make use of NIZK proofs to ensure both rogue ciphertext security and CCA security. We next give more details.

NIZK proofs for Rogue Ciphertext Security. Rogue ciphertext security ensures that an encryption of a message m always decrypts to the same message m, even if all ciphertexts in the batch and all committee members are controlled by a malicious adversary (Definition 8). There are four possible attack vectors through which an adversary could launch a rogue ciphertext attack against our construction. We address and mitigate each of these using tailored NIZK proofs and verification techniques.

Injecting malformed public keys. A malicious adversary could publish malformed public keys during the silent setup phase, causing the aggregation step of the decryption to return an incorrect message. Observe that, similar to [20], our public keys are verifiable as is, i.e., we do not need to introduce a special proof system. We can verify that a public key is sound by checking that $\mathsf{pk}_{u,j} \circ [1]_2 = \mathsf{pk}_{1,0} \circ [r_u \tau^j]_2$, for all $u \in [\ell], j \in [0, n-1]$ without sending a NIZK proof.

Issuing malformed partial decryptions. Malicious committee members could also attempt to issue malformed partial decryptions in order to manipulate the final decrypted message. We mitigate this attack by requiring committee members to prove the correctness of their partial decryption using a NIZK proof system, denoted $\mathsf{PS}_{\mathsf{pd}}$. The proof system is defined over statements $\chi := (\mathsf{crs}, \mathsf{pk}, \{\mathsf{ct}_i\}_{i\in[B]}, \mathsf{pd})$ and witnesses $\omega := \mathsf{sk}$, with the following relation: $\mathsf{R}_{\mathsf{pd}} = \big\{(\chi,\omega)\colon ([\mathsf{sk}]_1 = \mathsf{pk}_{1,0} \wedge \mathsf{pd} = \mathsf{sk} \cdot \sum_{i\in[B]} \mathsf{ct}_i.\mathsf{ct}_6\big\}$. This proof system can be similarly realized using a simple Schnorr proof. The proof size is 2 field elements, i.e., 64 bytes.

Inducing overflow in chunking. Recall that in our construction, the PRF key is decomposed into ℓ chunks of size $z_1 = \lambda_k/\ell$ bits, such that $k = \sum_{j\in[\ell]} k_j \cdot 2^{(j-1)z_1}$. We then encrypt the individual $[k_j]_T$ in Σ_{MSTE}. During decryption, we sum up each chunk across all ciphertexts in a batch and learn the aggregated chunk keys $[K_j]_T := [\sum_{i\in[B]} \mathsf{ct}_i.k_j]_T$ through STE decryption. Then, we run an algorithm to extract each k_j from $[K_j]_T$, which only works on the condition that $K_j \in [0, 2^{z_1+\log(B)} - 1]$.

Critically, an adversary could inject a ciphertext, where it samples a chunk key k_j outside of the permitted range. As a result, the discrete logarithm for the aggregated chunk key across all ciphertexts in the batch K_j would also be outside the permitted range causing the employed discrete logarithm algorithm to either fail entirely or not terminate in polynomial time.

Hence, we introduce range proof $\mathsf{PS}_{\mathsf{range}}$, forcing the encryptor to prove that each k_j is within the range $[0, 2^{z_1} - 1]$. For this, we can use an efficient range proof technique [70], [71], which requires us to add Pedersen commitments $\{\mathsf{com}_j\}_{j\in[\ell]}$ for each k_j to the ciphertext. The commitments and range proof add $\ell \cdot 6 \cdot \mathbb{G}_1$ elements to the ciphertext, which amounts to 2.25 KiB for $\ell = 8$. Note that we also need to prove that the $\{k_j\}_{j\in[\ell]}$ hidden within com_j are the same chunk keys that are encrypted within Σ_{MSTE}. We explain this in the next step.

Mismatching punctured keys and STE ciphertexts. An adversary could attempt the same attack described in [18], where mismatched keys are used between the punctured PRF key and the one encrypted within the ciphertext. Hence, we also need to prove that the chunking $\{k_j\}_{j\in[\ell]}$ that is (validly) encrypted in Σ_{MSTE}, is actually a valid decomposition of a key k matching the punctured key k^*. Additionally, to tie the range proofs to the ciphertext, we must also prove that the same chunk keys are hidden in the Pedersen commitments $\{\mathsf{com}_j\}_{j\in[\ell]}$.

To enforce this, we introduce a NIZK proof system $\mathsf{PS}_{\mathsf{cca}}$ for the relation $\mathsf{R}_{\mathsf{cca}}$ defined over statements $\chi = (\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in[n]}, \mathsf{ct} := (i, t, k^*, \beta, \mathsf{ct}_{\mathsf{MSTE}}, \{\mathsf{com}_j\}_{j\in[\ell]}, \pi^{\mathsf{range}}))$[5], and witness $\omega := (\{k_j\}_{j\in[\ell]}, \vec{s}, \{r_j^{\mathsf{com}}\}_{j\in[\ell]})$, such that

$$\mathsf{R}_{\mathsf{cca}} = \Bigg\{(\chi,\omega)\colon k^* = \mathsf{PRF.Punct}\Big(\sum_{j\in[\ell]} k_j \cdot 2^{(j-1)\cdot z}, i\Big) \wedge \mathsf{ct}_{\mathsf{MSTE}} = \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}\left(\{\mathsf{pk}_i\}_{i\in[n]}, \{k_j\}_{j\in[\ell]}, t; \vec{s}\right) \wedge \forall j \in [\ell]\colon \mathsf{com}_i = \mathsf{Ped.Commit}(k_j, r_j^{\mathsf{com}})\Bigg\}.$$

We instantiate $\mathsf{PS}_{\mathsf{cca}}$ with a Schnorr proof with a proof size of $2\ell + 6$ field elements, i.e., 704 bytes.

CCA Security. Finally, observe that the proof system $\mathsf{PS}_{\mathsf{cca}}$ is a proof of knowledge, which is straight-line simulation-extractable in the algebraic group model (AGM) [72], [73]. As a result, we can extract s_3 from its witness and use it to simulate the partial decryption oracle in the CCA security proof, ensuring CCA security for our construction.

5. We include the range proof within the statements to avoid ciphertext malleability and ensure CCA security.

7.2. Security

Theorem 12 (Security of Σ_{SB})**.** *The silent batched threshold encryption protocol* Σ_{SB} *is* SB-IND-CCA*-secure and rogue ciphertext secure.*

For a proof of Theorem 12, we refer to Appendix B.2 and B.3.

8. Experimental Evaluation

We experimentally evaluate our batched threshold encryption scheme with silent setup. We present our benchmarks alongside that of prior work [16], [17], [18], [19] building batched threshold encryption and threshold encryption with silent setup [20]. We emphasize that this is only to provide context because our results are incomparable to all prior work which either rely on an *interactive* setup or do not support batched decryption. In large scale decentralized systems such as Ethereum, where the committee in charge of voting for the next block changes every 12 seconds, a distributed key generation based setup is simply not feasible. Our implementation is available at https://github.com/guruvamsi-policharla/batched-silent-threshold-encryption.

Partial-Decryption Benchmarks (ms)		
Batch size	**[17], [19]**	**[18], This Work**
8	0.87	0.12
32	2.03	0.17
128	5.2	0.24
512	15.1	0.55

Figure 4. Time taken to compute partial decryptions with varying batch size using $n = 128$, parties with threshold $t = n/2$.

Reconstruction Benchmarks (s)			
Batch size	**[17], [19]**	**[18]**	**This Work**
8	0.06	0.05	4.5
32	0.17	0.56	4.6
128	0.65	7.9	11.8
512	3.02	125.5	129.5

Figure 5. Time taken to compute aggregate partial decryptions and recover messages with varying batch size using $n = 128$, parties with threshold $t = n/2$.

Setup. All of our experiments were run on a 2019 MacBook Pro with a 2.4 GHz Intel Core i9 processor and 16 GB of DDR4 RAM in single-threaded mode. We use `arkworks` [74] for implementations of pairing-friendly curves and their associated algebra, with benchmarks run over the BLS12-381 curve. We implement the CPA secure version of our scheme and compare against CPA secure versions of related work.

Encryption. Given an aggregated public key ek, encryption takes ≈ 16.4 ms independent of the batch size or the number of parties in the committee. Ciphertext sizes can be found in Fig. 6.

Partial Decryption. Decrypting a batch of ciphertexts in our scheme requires B group additions and one group exponentiation. This is similar to [18], but [17], [19] require a multi scalar multiplication of size B and a group exponentiation, making the partial decryption more expensive.

Reconstruction. Given sufficiently many partial decryptions, we run the combine algorithm of silent threshold encryption [20] to first recover the *aggregate* key of the batch of ciphertexts. However, the key is actually decomposed and stored in the exponent of the target group, requiring us to solve the discrete logarithm problem for small exponents. Once we have the aggregate key, we then need to run the combine algorithm from [18] to recover the messages – this is by far the most expensive step as it requires $O(B^2)$ number of pairings. In contrast [17], [19] only require $O(n \log^2 n)$ time,[6] but they come with the downside that messages must be encrypted to a particular "epoch" and they can only batch ciphertexts that have been encrypted to the same epoch. This results in a more complicated block-building strategy. We note that the optimization of introducing sub-batches in [18] simultaneously works for our scheme. Given the fact that we need to solve more discrete logarithms with more sub-batches, we can carefully choose the number of sub-batches to find the optimal setting. The partial decryption size grows linearly with the number of sub-batches. We provide benchmarks in Fig. 5. As we go towards smaller batch sizes, we observe the fixed cost of the silent threshold encryption aggregation (≈ 350 ms) and computing discrete logarithm (≈ 4 s).

Computing Discrete Logarithm. The strategy to solve the discrete logarithm for exponents smaller

6. Can be improved to $O(n \log n)$ with a smooth multiplicate domain such as the roots of unity of order n.

	Size (Elements)			Size (KiB)		
	Σ_{SB}	[18]	[17], [19]	Σ_{SB}	[18]	[17], [19]
crs	$[\ell \cdot (N+2) + B_{\max}] \cdot \mathbb{G}_1 +$ $\ell \cdot (N+3) \cdot \mathbb{G}_2 +$ $(2B_{\max} - 1) \cdot \mathbb{G}_2$	$B_{\max} \cdot \mathbb{G}_1 +$ $(2B_{\max} - 1) \cdot \mathbb{G}_2$	$B_{\max} \cdot \mathbb{G}_1 +$ $2 \cdot \mathbb{G}_2$	266.90	119.91	24.19
pk	$\ell \cdot N \cdot \mathbb{G}_1$	$1 \cdot \mathbb{G}_2$	$N \cdot \mathbb{G}_2$	48.00	0.09	6.00
ek	$6 \cdot \mathbb{G}_1 + 5 \cdot \mathbb{G}_2$	$1 \cdot \mathbb{G}_1 + 1 \cdot \mathbb{G}_2$	$1 \cdot \mathbb{G}_2 + 2$ bytes	0.75	0.14	0.10
ct	$(4+\ell) \cdot \mathbb{G}_1 + 5 \cdot \mathbb{G}_2$ $+ \log B_{\max}$	$4 \cdot \mathbb{G}_1 + 1 \cdot \mathbb{G}_2$ $+2$ bytes	$1 \cdot \mathbb{G}_1 + 3 \cdot \mathbb{G}_2$ $+2$ bytes	1.04	0.28	0.33
π^{ct}	$(2\ell + 6) \cdot \mathbb{F} + 6\ell \cdot \mathbb{G}_1$	$3 \cdot \mathbb{G}_1 + 2 \cdot \mathbb{F}$	$4 \cdot \mathbb{F}$	2.94	0.20	0.13
pd	$1 \cdot \mathbb{G}_1$	$1 \cdot \mathbb{G}_1$	$1 \cdot \mathbb{G}_1$	0.05	0.05	0.05
π^{pd}	$2 \cdot \mathbb{F}$	$2 \cdot \mathbb{F}$	$2 \cdot \mathbb{F}$	0.06	0.05	0.05

Figure 6. Sizes of various parameters and messages in our batched threshold encryption scheme Σ_{SB} with silent setup, and comparison between Σ_{SB} and [17], [18] (all of which require DKG setup). Concrete numbers are reported for $\ell = 8, B_{\max} = 512$, and $N = 128$.

than E is to first store 'markers' $\mathcal{M} = \{i * (E/M)\}_{i \in [0,M)}$ at M evenly spaced points in $[0, E)$ in a constant time look-up hash-map. Given the target instance t, we first compute $\mathcal{T} = \{t+i\}_{i \in [0, E/M)}$, and from the common element in $\mathcal{T} \cap \mathcal{M}$, we can recover the discrete logarithm. As a further storage optimization, we hash the markers using SHA256 and truncate to 6 bytes. E is chosen based on the number of chunks and batch size. Concretely, we set $E = 2^{41}$ and $M = 2^{25}$, which requires a storage of 448 MB. This allows us to support a maximum batch size of 2^9 with $\ell = 8$, and solving the discrete logarithm takes ≈ 500 ms. We emphasize that this is in single-thread mode, and this algorithm is trivially parallelizable. Furthermore, our implementation offers an effective time-space tradeoff by simply changing the number of markers stored.

9. Acknowledgements

The work of the fourth and the sixth authors is supported in part by the AFOSR Award FA9550-24-1-0156, and research grants from the Bakar Fund, J. P. Morgan Faculty Research Award, Supra Inc., Sui Foundation, and the Stellar Development Foundation. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of these institutes.

This work is funded by the European Research Council (ERC) under the European Union's Horizon 2020 and Horizon Europe research and innovation programs (grant CRYPTOLAYER-101044770). Furthermore, this research is supported by the German Federal Ministry of Education and Research, as well as the Hessen State Ministry for Higher Education, Research and the Arts, through their joint sponsorship of the National Research Center for Applied Cybersecurity ATHENE.

References

[1] S. Eskandari, S. Moosavi, and J. Clark, "Sok: Transparent dishonesty: front-running attacks on blockchain," in *International Conference on Financial Cryptography and Data Security*, 2019. 1

[2] L. Zhou, K. Qin, A. Cully, B. Livshits, and A. Gervais, "On the just-in-time discovery of profit-generating transactions in defi protocols," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021. 1

[3] Y. Wang, Y. Chen, S. Deng, and R. Wattenhofer, "Cyclic arbitrage in decentralized exchange markets," *Available at SSRN 3834535*, 2021. 1

[4] L. Zhou, K. Qin, C. F. Torres, D. V. Le, and A. Gervais, "High-frequency trading on decentralized on-chain exchanges," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021. 1

[5] G. Angeris, A. Evans, and T. Chitra, "A note on bundle profit maximization," *Stanford University*, 2021. 1

[6] K. Qin, L. Zhou, and A. Gervais, "Quantifying blockchain extractable value: How dark is the forest?" *arXiv preprint*, 2021. 1

[7] C. F. Torres, R. Camino *et al.*, "Frontrunner jones and the raiders of the dark forest: An empirical study of frontrunning on the ethereum blockchain," in *30th USENIX Security Symposium*, 2021. 1

[8] A. Judmayer, N. Stifter, P. Schindler, and E. R. Weippl, "Estimating (miner) extractable value is hard, let's go shopping!" *IACR Cryptology ePrint Archive*, 2021. 1

[9] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, "Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability," in *2020 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2020, pp. 910–927. 1, 4

[10] Flashbots, "Mev-boost," https://github.com/flashbots/mev-boost, 2022. 1, 4

[11] S. Dziembowski, S. Faust, and J. Luhn, "Shutter network: Private transactions from threshold cryptography," *IACR Cryptol. ePrint Arch.*, p. 1981, 2024. [Online]. Available: https://eprint.iacr.org/2024/1981 1, 2

[12] I. Bentov, Y. Ji, F. Zhang, L. Breidenbach, P. Daian, and A. Juels, "Tesseract: Real-time cryptocurrency exchange using trusted hardware," in *ACM CCS 2019*, L. Cavallaro, J. Kinder, X. Wang, and J. Katz, Eds. ACM Press, Nov. 2019, pp. 1521–1538. 1, 4

[13] M. Kelkar, F. Zhang, S. Goldfeder, and A. Juels, "Order-fairness for byzantine consensus," in *CRYPTO 2020, Part III*, ser. LNCS, D. Micciancio and T. Ristenpart, Eds., vol. 12172. Springer, Cham, Aug. 2020, pp. 451–480. 1, 4

[14] J. Bebel and D. Ojha, "Ferveo: Threshold decryption for mempool privacy in BFT networks," Cryptology ePrint Archive, Report 2022/898, 2022. [Online]. Available: https://eprint.iacr.org/2022/898 2, 4

[15] H. Zhang, L.-H. Merino, Z. Qu, M. Bastankhah, V. Estrada-Galiñanes, and B. Ford, "F3B: A Low-Overhead Blockchain Architecture with Per-Transaction Front-Running Protection," in *5th Conference on Advances in Financial Technologies (AFT 2023)*, vol. 282. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, pp. 3:1–3:23. 2, 4

[16] A. R. Choudhuri, S. Garg, J. Piet, and G.-V. Policharla, "Mempool privacy via batched threshold encryption: Attacks and defenses," in *USENIX Security 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, Aug. 2024. 2, 4, 15

[17] A. R. Choudhuri, S. Garg, G.-V. Policharla, and M. Wang, "Practical mempool privacy via one-time setup batched threshold encryption," in *USENIX Security Symposium*. USENIX Association, 2025. 2, 4, 15, 16

[18] J. Bormet, S. Faust, H. Othman, and Z. Qu, "BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention," in *USENIX Security Symposium*. USENIX Association, 2025. 2, 3, 4, 6, 7, 9, 10, 14, 15, 16, 24

[19] A. Agarwal, R. Fernando, and B. Pinkas, "Efficiently-thresholdizable batched identity based encryption, with applications," Cryptology ePrint Archive, Paper 2024/1575, 2024. [Online]. Available: https://eprint.iacr.org/2024/1575 2, 4, 15, 16

[20] S. Garg, D. Kolonelos, G.-V. Policharla, and M. Wang, "Threshold encryption with silent setup," in *Advances in Cryptology – CRYPTO 2024*, L. Reyzin and D. Stebila, Eds. Cham: Springer Nature Switzerland, 2024, pp. 352–386. 2, 3, 4, 6, 7, 8, 9, 11, 14, 15, 20, 21

[21] Shutter Network contributors, "The shutter network," https://shutter.network, 2021. 4

[22] P. Momeni, S. Gorbunov, and B. Zhang, "Fairblock: Preventing blockchain front-running with minimal overheads," in *SecureComm*, ser. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 462. Springer, 2022, pp. 250–271. 4

[23] N. Döttling, L. Hanzlik, B. Magri, and S. Wohnig, "McFly: Verifiable encryption to the future made practical," in *FC 2023, Part I*, ser. LNCS, F. Baldimtsi and C. Cachin, Eds., vol. 13950. Springer, Cham, May 2023, pp. 252–269. 4

[24] S. Suegami, S. Ashizawa, and K. Shibano, "Constant-cost batched partial decryption in threshold encryption," Cryptology ePrint Archive, Paper 2024/762, 2024. [Online]. Available: https://eprint.iacr.org/2024/762 4

[25] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch, "Verifiable delay functions," in *CRYPTO 2018, Part I*, ser. LNCS, H. Shacham and A. Boldyreva, Eds., vol. 10991. Springer, Cham, Aug. 2018, pp. 757–788. 4

[26] J. Liu, T. Jager, S. A. Kakvi, and B. Warinschi, "How to build time-lock encryption," *DCC*, vol. 86, no. 11, pp. 2549–2586, 2018. 4

[27] G. Avitabile, N. Döttling, B. Magri, C. Sakkas, and S. Wohnig, "Signature-based witness encryption with compact ciphertext," Cryptology ePrint Archive, Paper 2024/1477, 2024. [Online]. Available: https://eprint.iacr.org/2024/1477 4

[28] S. Garg, C. Gentry, A. Sahai, and B. Waters, "Witness encryption and its applications," in *45th ACM STOC*, D. Boneh, T. Roughgarden, and J. Feigenbaum, Eds. ACM Press, Jun. 2013, pp. 467–476. 4

[29] L. Devadas, A. Jain, B. Waters, and D. J. Wu, "Succinct witness encryption for batch languages and applications," Cryptology ePrint Archive, Paper 2025/944, 2025. [Online]. Available: https://eprint.iacr.org/2025/944 4

[30] C. Stathakopoulou, S. Rüsch, M. Brandenburger, and M. Vukolić, "Adding fairness to order: Preventing front-running attacks in bft protocols using tees," in *2021 40th International Symposium on Reliable Distributed Systems (SRDS)*, 2021, pp. 34–45. 4

[31] H. Ragab, A. Milburn, K. Razavi, H. Bos, and C. Giuffrida, "CrossTalk: Speculative data leaks across cores are real," in *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021, pp. 1852–1867. 4

[32] J. Van Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the keys to the intel SGX kingdom with transient out-of-order execution," in *USENIX Security 2018*, W. Enck and A. P. Felt, Eds. USENIX Association, Aug. 2018, pp. 991–1008. 4

[33] K. Kursawe, "Wendy, the good little fairness widget: Achieving order fairness for blockchains," in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, 2020, pp. 25–36. 4

[34] M. Kelkar, S. Deb, S. Long, A. Juels, and S. Kannan, "Themis: Fast, strong order-fairness in byzantine consensus," in *ACM CCS 2023*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM Press, Nov. 2023, pp. 475–489. 4

[35] C. McMenamin, V. Daza, M. Fitzi, and P. O'Donoghue, "Fairtradex: A decentralised exchange preventing value extraction," in *Proceedings of the 2022 ACM CCS Workshop on Decentralized Finance and Security*, ser. DeFi'22, 2022. 4

[36] L. Heimbach and R. Wattenhofer, "Eliminating sandwich attacks with the help of game theory," in *ASIACCS 22*, Y. Suga, K. Sakurai, X. Ding, and K. Sako, Eds. ACM Press, May / Jun. 2022, pp. 153–167. 4

[37] R. Gennaro, S. Jarecki, H. Krawczyk, and T. Rabin, "Secure distributed key generation for discrete-log based cryptosystems," in *EUROCRYPT'99*, ser. LNCS, J. Stern, Ed., vol. 1592. Springer, Berlin, Heidelberg, May 1999, pp. 295–310. 4

[38] J. Groth, "Non-interactive distributed key generation and key resharing," Cryptology ePrint Archive, Report 2021/339, 2021. [Online]. Available: https://eprint.iacr.org/2021/339 4

[39] A. Boldyreva, "Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme," in *PKC 2003*, ser. LNCS, Y. Desmedt, Ed., vol. 2567. Springer, Berlin, Heidelberg, Jan. 2003, pp. 31–46. 4

[40] F. Baldimtsi, K. K. Chalkias, F. Garillot, J. Lindstrom, B. Riva, A. Roy, M. Sedaghat, A. Sonnino, P. Waiwitlikhit, and J. Wang, "Subset-optimized BLS multi-signature with key aggregation," Cryptology ePrint Archive, Report 2023/498, 2023. [Online]. Available: https://eprint.iacr.org/2023/498 4

[41] J. Groth, "On the size of pairing-based non-interactive arguments," in *EUROCRYPT 2016, Part II*, ser. LNCS, M. Fischlin and J.-S. Coron, Eds., vol. 9666. Springer, Berlin, Heidelberg, May 2016, pp. 305–326. 4

[42] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Report 2019/953, 2019. [Online]. Available: https://eprint.iacr.org/2019/953 4

[43] S. Micali, L. Reyzin, G. Vlachos, R. S. Wahby, and N. Zeldovich, "Compact certificates of collective knowledge," in *2021 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2021, pp. 626–641. 4

[44] S. Garg, A. Jain, P. Mukherjee, R. Sinha, M. Wang, and Y. Zhang, "hinTS: Threshold signatures with silent setup," Cryptology ePrint Archive, Report 2023/567, 2023. [Online]. Available: https://eprint.iacr.org/2023/567 4

[45] S. Das, P. Camacho, Z. Xiang, J. Nieto, B. Bünz, and L. Ren, "Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold," in *ACM CCS 2023*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM Press, Nov. 2023, pp. 356–370. 4

[46] C. Freitag, B. Waters, and D. J. Wu, "How to use (plain) witness encryption: Registered ABE, flexible broadcast, and more," in *CRYPTO 2023, Part IV*, ser. LNCS, H. Handschuh and A. Lysyanskaya, Eds., vol. 14084. Springer, Cham, Aug. 2023, pp. 498–531. 4

[47] D. Kolonelos, G. Malavolta, and H. Wee, "Distributed broadcast encryption from bilinear groups," in *ASIACRYPT 2023, Part V*, ser. LNCS, J. Guo and R. Steinfeld, Eds., vol. 14442. Springer, Singapore, Dec. 2023, pp. 407–441. 4

[48] R. Garg, G. Lu, B. Waters, and D. J. Wu, "Realizing flexible broadcast encryption: How to broadcast to a public-key directory," in *ACM CCS 2023*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM Press, Nov. 2023, pp. 1093–1107. 4

[49] S. Garg, M. Hajiabadi, M. Mahmoody, and A. Rahimi, "Registration-based encryption: Removing private-key generator from IBE," in *TCC 2018, Part I*, ser. LNCS, A. Beimel and S. Dziembowski, Eds., vol. 11239. Springer, Cham, Nov. 2018, pp. 689–718. 4

[50] S. Garg, M. Hajiabadi, M. Mahmoody, A. Rahimi, and S. Sekar, "Registration-based encryption from standard assumptions," in *PKC 2019, Part II*, ser. LNCS, D. Lin and K. Sako, Eds., vol. 11443. Springer, Cham, Apr. 2019, pp. 63–93. 4

[51] N. Glaeser, D. Kolonelos, G. Malavolta, and A. Rahimi, "Efficient registration-based encryption," in *ACM CCS 2023*, W. Meng, C. D. Jensen, C. Cremers, and E. Kirda, Eds. ACM Press, Nov. 2023, pp. 1065–1079. 4

[52] S. Hohenberger, G. Lu, B. Waters, and D. J. Wu, "Registered attribute-based encryption," in *EUROCRYPT 2023, Part III*, ser. LNCS, C. Hazay and M. Stam, Eds., vol. 14006. Springer, Cham, Apr. 2023, pp. 511–542. 4

[53] Z. Zhu, K. Zhang, J. Gong, and H. Qian, "Registered ABE via predicate encodings," Cryptology ePrint Archive, Report 2023/1383, 2023. [Online]. Available: https://eprint.iacr.org/2023/1383 4

[54] D. Francati, D. Friolo, M. Maitra, G. Malavolta, A. Rahimi, and D. Venturi, "Registered (inner-product) functional encryption," in *ASIACRYPT 2023, Part V*, ser. LNCS, J. Guo and R. Steinfeld, Eds., vol. 14442. Springer, Singapore, Dec. 2023, pp. 98–133. 4

[55] Z. Zhu, J. Li, K. Zhang, J. Gong, and H. Qian, "Registered functional encryptions from pairings," in *EUROCRYPT 2024, Part II*, ser. LNCS, M. Joye and G. Leander, Eds., vol. 14652. Springer, Cham, May 2024, pp. 373–402. 4

[56] L. Reyzin, A. Smith, and S. Yakoubov, "Turning hate into love: Compact homomorphic ad hoc threshold encryption for scalable mpc," in *Cyber Security Cryptography and Machine Learning*, 2021. 4

[57] E. Ben-Sasson, A. Chiesa, M. Riabzev, N. Spooner, M. Virza, and N. P. Ward, "Aurora: Transparent succinct arguments for R1CS," in *EUROCRYPT 2019, Part I*, ser. LNCS, Y. Ishai and V. Rijmen, Eds., vol. 11476. Springer, Cham, May 2019, pp. 103–128. 5

[58] C. Ràfols and A. Zapico, "An algebraic framework for universal and updatable SNARKs," in *CRYPTO 2021, Part I*, ser. LNCS, T. Malkin and C. Peikert, Eds., vol. 12825. Virtual Event: Springer, Cham, Aug. 2021, pp. 774–804. 5

[59] M. Campanelli, A. Nitulescu, C. Ràfols, A. Zacharakis, and A. Zapico, "Linear-map vector commitments and their practical applications," in *ASIACRYPT 2022, Part IV*, ser. LNCS, S. Agrawal and D. Lin, Eds., vol. 13794. Springer, Cham, Dec. 2022, pp. 189–219. 5

[60] V. Shoup, "Lower bounds for discrete logarithms and related problems," in *EUROCRYPT'97*, ser. LNCS, W. Fumy, Ed., vol. 1233. Springer, Berlin, Heidelberg, May 1997, pp. 256–266. 5, 21

[61] U. M. Maurer, "Abstract models of computation in cryptography (invited paper)," in *10th IMA International Conference on Cryptography and Coding*, ser. LNCS, N. P. Smart, Ed., vol. 3796. Springer, Berlin, Heidelberg, Dec. 2005, pp. 1–12. 5, 21

[62] D. Boneh, X. Boyen, and E.-J. Goh, "Hierarchical identity based encryption with constant size ciphertext," in *EUROCRYPT 2005*, ser. LNCS, R. Cramer, Ed., vol. 3494. Springer, Berlin, Heidelberg, May 2005, pp. 440–456. 5, 19

[63] X. Boyen, "The uber-assumption family (invited talk)," in *PAIRING 2008*, ser. LNCS, S. D. Galbraith and K. G. Paterson, Eds., vol. 5209. Springer, Berlin, Heidelberg, Sep. 2008, pp. 39–56. 5, 19

[64] J. Dujmovic, R. Garg, and G. Malavolta, "Time-lock puzzles with efficient batch solving," in *EUROCRYPT 2024, Part II*, ser. LNCS, M. Joye and G. Leander, Eds., vol. 14652. Springer, Cham, May 2024, pp. 311–341. 6

[65] A. Kate, G. M. Zaverucha, and I. Goldberg, "Constant-size commitments to polynomials and their applications," in *ASIACRYPT 2010*, ser. LNCS, M. Abe, Ed., vol. 6477. Springer, Berlin, Heidelberg, Dec. 2010, pp. 177–194. 8

[66] J. M. Pollard, "Monte carlo methods for index computation (mod p)," *Mathematics of computation*, vol. 32, no. 143, pp. 918–924, 1978. 11

[67] E. Teske, *Speeding up Pollard's rho method for computing discrete logarithms*. Springer, 1998. 11

[68] H. Cohen, *A course in computational algebraic number theory*. Springer Science & Business Media, 2013, vol. 138. 11

[69] K. Kurosawa, "Multi-recipient public-key encryption with shortened ciphertext," in *PKC 2002*, ser. LNCS, D. Naccache and P. Paillier, Eds., vol. 2274. Springer, Berlin, Heidelberg, Feb. 2002, pp. 48–63. 11

[70] D. Boneh, B. Fisch, A. Gabizon, and Z. Williamson, "A simple range proof from polynomial commitments (2020)," *URL: https://hackmd.io/@dabo/B1U4kx8XI*, 2020. 14

[71] D. Boneh, J. Drake, B. Fisch, and A. Gabizon, "Efficient polynomial commitment schemes for multiple points and polynomials," Cryptology ePrint Archive, Report 2020/081, 2020. [Online]. Available: https://eprint.iacr.org/2020/081 14

[72] G. Fuchsbauer, E. Kiltz, and J. Loss, "The algebraic group model and its applications," in *CRYPTO 2018, Part II*, ser. LNCS, H. Shacham and A. Boldyreva, Eds., vol. 10992. Springer, Cham, Aug. 2018, pp. 33–62. 14

[73] G. Fuchsbauer, A. Plouviez, and Y. Seurin, "Blind schnorr signatures and signed ElGamal encryption in the algebraic group model," in *EUROCRYPT 2020, Part II*, ser. LNCS, A. Canteaut and Y. Ishai, Eds., vol. 12106. Springer, Cham, May 2020, pp. 63–95. 14

[74] arkworks contributors, "`arkworks` zksnark ecosystem," https://arkworks.rs, 2022. 15

Appendix A. Additional Preliminaries

In the following, we present some theorems, lemmas, and definitions for cryptographic building blocks we use in our constructions.

A.1. Master Theorem

Theorem 13 (Master Theorem [62], [63]). *Let $p \in \mathbb{N}$ be a prime, let $mnv_1, v_2, v_t \in \mathbb{N}$, let $\mathbf{L}_1 \in \mathbb{F}_p[X_1, \ldots, X_n]^{v_1}$, $\mathbf{L}_2 \in \mathbb{F}_p[X_1, X_n]^{v_2}$, $\mathbf{L}_T \in \mathbb{F}_p[X_1, \ldots X_n]^{v_n}$, and $f \in \mathbb{F}_p[X_1, \ldots, X_n]$ be tuples of n-variate polynomials of maximum degree $d_{\mathbf{L}_1}, d_{\mathbf{L}_2}$, $d_{\mathbf{L}_T}$, and d_f respectively. Denote $d = \max(d_{\mathbf{L}_1} + d_{\mathbf{L}_2}, d_{\mathbf{L}_T}, d_f)$, and $v = v_1 + v_2 + v_t$. If for all $j \in [v_f]$ it holds that f_j is independent of $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$, then for any generic adversary $\mathcal{A}$ that makes at most q group oracle queries:*

$$\left| \Pr\left[\mathcal{A}\begin{pmatrix} p, \mathsf{h}_1[\mathbf{L}_1(x)], \\ \mathsf{h}_2[\mathbf{L}_2(x)], \mathsf{h}_T[\mathbf{L}_T(x), f(x)] \end{pmatrix} = 1\right] - \Pr\left[\mathcal{A}\begin{pmatrix} p, \mathsf{h}_1[\mathbf{L}_1(x)], \\ \mathsf{h}_2[\mathbf{L}_2(x)], \mathsf{h}_T[\mathbf{L}_T(x), r] \end{pmatrix} = 1\right] \right| \leq v_f \cdot \frac{(q+v+2)^2 \cdot d}{2p},$$

where $\mathsf{h}_1, \mathsf{h}_2$, and h_T denote the corresponding handles, and the probabilities are taken over the choice of $x \leftarrow\!\$\ \mathbb{F}_p^n$ and $r \leftarrow\!\$\ \mathbb{F}_p$.

Definition 14 (Independence). *Let the completion $\mathbf{C}(\mathbf{L}) = \{\mathbf{L}_1 \otimes \mathbf{L}_2\} \cup \mathbf{L}_T$ of size D for lists $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$.*

$$\mathbf{C}(\mathbf{L}) = \{g_1(X_1, \ldots, X_n), \ldots g_D(X_1, \ldots, X_n)\}$$

We say that an element f is dependent on lists $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$, if there exist coefficients $\kappa_i \in \mathbb{F}_p$ such that

$$f(X_1, \ldots, X_n) = \sum_{i=1}^{D} \kappa_i \cdot g_i(X_1, \ldots, X_N).$$

Otherwise, we say that f is independent of $(\mathbf{L}_1, \mathbf{L}_2, \mathbf{L}_T)$.

A.2. Commitment Schemes

Definition 15 (Commitment Schemes). *A commitment scheme Σ for message space $\mathcal{M}$ and randomness space $\mathcal{R}$ is a tuple of algorithms $\Sigma = (\mathsf{Setup}, \mathsf{Commit}, \mathsf{Vfy})$ such that:*

- $\mathsf{pp} \leftarrow\!\$\ \mathsf{Setup}(1^\lambda)$. *Given the security parameter λ, Setup generates public parameters pp that are implicit input to all subsequent algorithms.*
- $\mathsf{com} \leftarrow \mathsf{Commit}(m; r)$. *Given a message m and randomness r, the Commit algorithm generates a commitment com.*
- $1/0 \leftarrow \mathsf{Vfy}(\mathsf{com}, m, r)$. *The verification algorithm returns 1 if the decommitment r opens the commitment com to message m and 0 otherwise.*

A commitment scheme must satisfy the following properties:

- ***Correctness.*** *For all messages $m \in \mathcal{M}$ and randomness $r \in \mathcal{R}$ it holds that $\mathsf{Vfy}(\mathsf{Commit}(m; r), m, r) = 1$.*
- ***Perfect Hiding.*** *For all unbounded adversaries $\mathcal{A}$, all $\lambda \in \mathbb{N}$, all $\mathsf{pp} \in \mathsf{Setup}(1^\lambda)$ and all messages $m_0, m_1 \in \mathcal{M}$, it holds that*

$$\Pr[\mathcal{A}(\mathsf{pp}, m_0, m_1, \mathsf{Commit}(m_0)) = 1] = \Pr[\mathcal{A}(\mathsf{pp}, m_0, m_1, \mathsf{Commit}(m_1)) = 1],$$

- ***Computational Binding.*** *For all PPT adversaries $\mathcal{A}$ there exists a negligible function* negl *such that*

$$\Pr\left[\begin{array}{c} m \neq m' \wedge \\ \mathsf{Vfy}(\mathsf{com}, m, r) \\ \wedge \mathsf{Vfy}(\mathsf{com}, m', r') \end{array} : \begin{array}{c} \mathsf{pp} \leftarrow\$ \, \mathsf{Setup}(1^\lambda) \\ (\mathsf{com}, m, r, m', r') \\ \leftarrow\$ \, \mathcal{A}(1^\lambda, \mathsf{pp}) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

A.3. NIZK PoKs

Definition 16 (NIZK-PoK)**.** *Let* R *be an NP-relation of statement-witness pairs* $(\chi, \omega) \in \mathsf{R}$ *a non-interactive zero-knowledge proof system of knowledge* PS_R *for* R *is a tuple of algorithms* $\mathsf{PS}_\mathsf{R} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Vfy}, \mathsf{Sim}, \mathsf{Ext})$ *with the following syntax:*

- $(\mathsf{crs}, \mathsf{td}) \leftarrow\$ \, \mathsf{Setup}(1^\lambda)$. *Given the security parameter λ, the* Setup *algorithm outputs a common reference string* crs *and a simulation and extraction trapdoor* td. *The* crs *is implicit input to all subsequent algorithms.*
- $\pi \leftarrow\$ \, \mathsf{Prove}(\chi, \omega)$. *For a statement-witness pair (χ, ω), the* Prove *algorithm returns a proof π.*
- $1/0 \leftarrow \mathsf{Vfy}(\chi, \pi)$. *Given a statement χ and a proof π, the* Vfy *algorithm outputs 1, iff π is valid for χ.*
- $\pi \leftarrow\$ \, \mathsf{Sim}(\mathsf{td}, \chi)$. *Given the trapdoor* td *and a statement χ, the simulation algorithm* Sim *outputs a simulated proof π.*
- $w \leftarrow\$ \, \mathsf{Ext}(\mathsf{td}, \chi, \pi)$. *Given the trapdoor* td*, a statement χ and a proof π, the extractor* Ext *returns a witness ω.*

A proof system PS_R *must fulfill the following properties.*

- *Correctness: For all* $(\chi, \omega) \in \mathsf{R}$ *it holds that* $\mathsf{Vfy}(\chi, \mathsf{Prove}(\chi, \omega)) = 1$.
- *Simulation-Extractability: For all PPT adversaries $\mathcal{A}$, there exists a negligible function* negl *such that*

$$\Pr\left[\begin{array}{c} \mathsf{Vfy}(\chi, \pi) = 1 \\ \wedge\ (\chi, \omega) \notin \mathsf{R} \\ \wedge \chi \notin Q \end{array} : \begin{array}{c} (\mathsf{crs}, \mathsf{td}) \leftarrow\$ \, \mathsf{Setup}(1^\lambda) \\ (\chi, \pi) \leftarrow\$ \, \mathcal{A}^{\mathsf{Sim}(\mathsf{td}, \cdot)}(1^\lambda, \mathsf{crs}) \\ \omega \leftarrow\$ \, \mathsf{Ext}(\mathsf{td}, \chi, \pi) \end{array}\right] \leq \mathsf{negl}(\lambda),$$

  where Q is the set of $\mathcal{A}$'s queries to the $\mathsf{Sim}(\mathsf{td}, \cdot)$ *oracle.*

- *Knowledge-Soundness: For all PPT adversaries $\mathcal{A}$, there exists a negligible function* negl *such that*

$$\Pr\left[\begin{array}{c} \mathsf{Vfy}(\chi, \pi) = 1 \\ \wedge\ (\chi, \omega) \notin \mathsf{R} \end{array} : \begin{array}{c} (\mathsf{crs}, \mathsf{td}) \leftarrow\$ \, \mathsf{Setup}(1^\lambda) \\ (\chi, \pi) \leftarrow\$ \, \mathcal{A}(1^\lambda, \mathsf{crs}) \\ \omega \leftarrow\$ \, \mathsf{Ext}(\mathsf{td}, \chi, \pi) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

  Note that knowledge-soundness is a weaker version of simulation-extractability, where the adversary does not get access to simulated ciphertexts.
- *Zero-Knowledge: For all* $(\chi, \omega) \in \mathsf{R}$ *it holds that*

$$(\mathsf{crs}, \chi, \mathsf{Prove}(\chi, \omega)) \approx_c (\mathsf{crs}, \chi, \mathsf{Sim}(\mathsf{td}, \chi)),$$

  where $(\mathsf{crs}, \mathsf{td}) \leftarrow\$ \, \mathsf{Setup}(1^\lambda)$.

A.4. Key-Homomorphic Puncturable PRF

Definition 17 (Pseudorandomnes of PRF)**.** *A puncturable* PRF *is pseudorandom if for all PPT adversaries $\mathcal{A}$, there exists a negligible function* negl *such that*

$$\Pr[\mathsf{Game\text{-}Pr}_\mathcal{A}^\mathsf{PRF}(1^\lambda) = 1] \leq 1/2 + \mathsf{negl}(\lambda),$$

where $\mathsf{Game\text{-}Pr}$ *is defined in Figure 7.*

$\mathsf{Game\text{-}Pr}_\mathcal{A}(1^\lambda)$

$(1^n, \mathsf{st}) \leftarrow\$ \, \mathcal{A}(1^\lambda)$

$\mathsf{pp} \leftarrow\$ \, \mathsf{Setup}(1^\lambda, 1^n)$

$(i^*, \mathsf{st}) \leftarrow\$ \, \mathcal{A}(\mathsf{st}, \mathsf{pp})$

$k \leftarrow\$ \, \mathsf{KGen}(1^\lambda); k^* \leftarrow\$ \, \mathsf{Punct}(k, i^*); b \leftarrow\$ \, \{0,1\}$

if $b = 0$ **then** $y \leftarrow\$ \, \mathcal{Y}$ **else** $y \leftarrow \mathsf{Eval}(k, i^*)$

$b' \leftarrow\$ \, \mathcal{A}(\mathsf{st}, k^*, y)$

Figure 7. Pseudorandomness game of PRF where $\mathcal{Y}$ denotes the evaluation space of the PRF.

For the sake of completion, we also recall the semantic security definition for silent setup encryption (without batching), following [20].

Definition 18 (S-IND-CPA-security)**.** *An STE scheme Σ is* S-IND-CPA*-secure if for all PPT-adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3)$ there exists a negligible function* negl *such that*

$$\Pr[\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_\mathcal{A}^\Sigma(1^\lambda) = 1] \leq \frac{1}{2} + \mathsf{negl}(\lambda),$$

where $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}$ *is defined in Figure 1.*

Appendix B. Security

B.1. Semantic Security of Σ_{MSTE}

Proof of Lemma 11. Our proof follows the technique from [20], but accounts for the fact that we encrypt ℓ chunks instead of a single message. To do so, we prove in the GGM [60], [61] that the following two games are computationally indistinguishable.

Note that in our proof we consider a variant of our construction, where we generate a random generator $[r_j]_{1,2}$ for *all* chunks $j \in [\ell]$ instead of setting $r_1 = 1$. This analysis carries over to the Σ_{MSTE} construction detailed in Section 6.4.

Game_0: Game_0 is equivalent to the Game-S-IND-CPA security game (Figure 2), instantiated with Σ_{MSTE}. In Game_0, the challenger first samples $\tau, \gamma, r_1, \ldots, r_\ell \leftarrow\$ \mathbb{F}^{\ell+2}$ and computes the crs as $\{[r_m]_{1,2}, [r_m\gamma]_2, [r_m\tau]_{1,2}, \ldots, [r_m\tau^n]_{1,2}\}_{m\in[\ell]}$. The challenger sends the corresponding handles to the adversary $\mathcal{A}$, who responds with a universe $\mathcal{C} \subseteq [N]$, a set of corrupted parties $\mathsf{Cor} \subseteq \mathcal{C}$, and a threshold $t > |\mathsf{Cor}|$. Then, the challenger computes $(\mathsf{sk}_i, \mathsf{pk}_i) \leftarrow\$ \mathsf{KGen}(\mathsf{pp})$ for all $i \in \mathcal{H}$, where $\mathcal{H} = \mathcal{C} \setminus \mathsf{Cor}$ and sends the public keys $\{\mathsf{pk}_i\}_{i\in\mathcal{H}}$ to $\mathcal{A}$. We define $\mathcal{H}^* = \mathcal{H} \cup \{0\}$. To compute the public key of the corrupted parties, the adversary needs to query the GGM oracle with all $\{sk_i\}_{i\in\mathsf{Cor}}$, receiving handles to the corresponding public keys in return. Then, $\mathcal{A}$ sends $m^{(0)}$ and $m^{(1)}$. The challenger samples a random bit b and computes $\mathsf{ct}^* \leftarrow\$ \mathsf{Enc}(\mathsf{pp}, \{\mathsf{pk}_i\}_{i\in\mathcal{C}}, t, m_b)$, sending the handles to elements in ct^* to $\mathcal{A}$. The view of the adversary now consists of the tuple $(N, t, \mathcal{C}, \{\mathsf{sk}_i\}_{i\in\mathsf{Cor}})$ and handles to $(\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in\mathcal{C}}, \mathsf{ct}^*)$. Finally, the adversary returns a bit b' and wins whenever $b = b'$. Throughout the course of the game, $\mathcal{A}$ can query the challenger to make arbitrary generic group operations.

Game_1: In Game_1, we modify how $\mathsf{ct}_{9,i}$ is computed in the challenge ciphertext. Instead of padding the message $[r_i m_i^{(b)}]_T$ with $[r_i s_5]_T$ for all $i \in [\ell]$, the challenger samples uniformly random $f_i \leftarrow\$ \mathbb{Z}_p$ and sets $\mathsf{ct}_{9,i} = [v_i \cdot (f_i + m_i^{(b)} r_i)]_1$, effectively replacing $r_i s_5$ with f_i.

Claim 19. *It holds that* $\mathsf{Game}_0 \approx_c \mathsf{Game}_1$ *in the generic group model.*

Proof of Claim 19. For a matrix A, we denote with $A^{[\ell]}$ the matrix that results from repeating each column of A a total of ℓ times. Hence, if A has dimensions $u \times v$, then $A^{[\ell]}$ has dimensions $u \times \ell v$. We denote as $\mathbf{a}_i$ the i-th column of A and as $\mathbf{b}_i$ the i-th column of B. Further, we write $B_{\neq j} = (\mathbf{b_1}, \ldots, \mathbf{b}_{j-1}, \mathbf{b}_{j+1}, \ldots, \mathbf{b}_\ell)$ to denote the matrix B without column j. Given two matrices A and B, we write $A|B$ to denote concatenation of the columns of A and B. We denote the formal variables as $\mathbf{K} = \{K_i\}_{i\in\mathcal{H}}$, $\mathbf{S} = (S_1, \ldots, S_5)$, $\mathbf{R} = \{R_i\}_{i\in[\ell]}$, $\mathbf{V} = \{V_i\}_{i\in[\ell]}$, and $\mathbf{Y} = (X, \mathbf{K}, \Gamma, \mathbf{S}, \mathbf{R}, \mathbf{V})$, corresponding to the secret elements τ, $\{\mathsf{sk}_i\}_{i\in\mathcal{H}}$, γ, $\mathbf{s}$, $\{r_i\}_{i\in[\ell]}$, and $\{v_i\}_{i\in[\ell]}$.

Inspecting the view of the adversary, we define the following lists of polynomials over the handles known to the adversary. For ease of notation, we denote $R_0 := 1$

$$\mathbf{L}_1(\mathbf{Y}) = \Bigg(\overbrace{\{R_m, R_m X, \ldots, R_m X^N\}}^{\mathsf{crs}}{}_{m\in[0,\ell]},$$

$$\overbrace{\{R_m K_i, R_m K_i X, \ldots, R_m K_i X^{N-1}\}}^{\{\mathsf{pk}_i\}_{i\in\mathcal{H}}}{}_{i\in\mathcal{H}, m\in[0,\ell]},$$

$$\overbrace{S_1 \left(\sum_{i\in\mathcal{H}} K_i L_i(X) + \sum_{i\in\mathsf{Cor}} \mathsf{sk}_i L_i(X) \right) + S_4 X^t + S_5}^{\mathbf{S}^\intercal \mathbf{a}_1},$$

$$\underbrace{S_3}_{\mathbf{S}^\intercal\mathbf{a}_6}, \underbrace{S_5 X}_{\mathbf{S}^\intercal\mathbf{a}_8}, \{V_m R_m\}_{m\in[\ell]}, [V_m R_m S_5]_{m\in[\ell]} \Bigg)$$

$$\mathbf{L}_2(\mathbf{Y}) = \Bigg(\underbrace{\{R_m, R_m\Gamma, R_m X, \ldots, R_m X^N\}_{m\in[0,\ell]}}_{\mathsf{crs}},$$

$$\underbrace{S_1 + S_3\Gamma}_{\mathbf{S}^\intercal\mathbf{a}_2}, \underbrace{S_1(X^N - 1)}_{\mathbf{S}^\intercal\mathbf{a}_3}, \underbrace{X S_1 + X^2 S_2}_{\mathbf{S}^\intercal\mathbf{a}_4}, \underbrace{S_2}_{\mathbf{S}^\intercal\mathbf{a}_5}, \underbrace{S_4}_{\mathbf{S}^\intercal\mathbf{a}_7},$$

$$\{V_m^{-1}\}_{m\in[\ell]} \Bigg)$$

$$\mathbf{L}_T(\mathbf{Y}) = (1)$$
$$f_j(\mathbf{Y}) = R_j S_5 \quad \forall j \in [\ell]$$

Ultimately, we want to show that for all $j \in [\ell]$, it holds that $f_j(\mathbf{Y})$ is independent of $F_j = (\mathbf{C}(\mathbf{L}) \setminus \{V_j R_j S_5 \cdot V_j^{-1}\}) \cup \{R_i S_5\}_{i\in[\ell], i\neq j}$[7], where $\mathbf{C}$ refers to the completion. Henceforth, we denote $D = |F_j|$

To do so, we first show that, if f_j depends on F_j, then there exists a vector of polynomials $\mathbf{g} \in (\mathbb{Z}_p[X, \mathbf{K}, \Gamma, \mathbf{R}])^{8\ell+\ell-1}$ that does not depend

7. Here, we remove the trivial combination of $V_j R_j S_5 \cdot V_j^{-1}$ to account for the fact that the adversary must compute $R_j S_5$ in some other way than pairing $\mathsf{ct}_{9,j}$ with $[v_j^{-1}]_2$ to distinguish f_j from random, as the factor $R_j S_5$ is replaced with f_j in Game_1.

on $\mathbf{S}$ and $\mathbf{V}$ such that $\mathbf{S}^\intercal(A^{[\ell]}|B_{\neq j}) \cdot \mathbf{g} = \mathbf{S}^\intercal \mathbf{b}_j$. Since $f_j(\mathbf{Y}) = R_j S_5$, it can only be dependent on F_j if there exists a linear combination $R_j S_5 = \sum_{i\in[D]} \kappa_i q_i(\mathbf{Y})$, where q_i are the elements of $F_j = \bigcup q_i$. We divide F_j into three types:

1) Terms that do not involve $\mathbf{S}$. E.g., $(K_1 X^{N-1}) \cdot X$.
2) Terms that involve $\mathbf{S}$ on both sides. E.g., $S_3 \cdot (S_1 + S_3\Gamma)$.
3) Terms that involve $\mathbf{S}$ on exactly one side. E.g., $S_3 \cdot X$.

Since the first two types of elements can not symbolically play a role in the linear combination $R_j S_5 = \sum_{i\in[D]} \kappa_i q_i(\mathbf{Y})$, we are only left with the third type of elements. Additionally, we observe that elements containing V_m or V_m^{-1} can not symbolically play any role in a combination of $R_j S_5$, which leaves us with

$$\mathbf{S}^\intercal(A^{[\ell]}|B_{\neq j}) \cdot \mathbf{g}(X, \mathbf{K}, \Gamma, \mathbf{R}) = \mathbf{S}^\intercal \mathbf{b}_j.$$

As $\mathbf{S}$ appears on both sides, we conclude that it must symbolically hold that

$$(A^{[\ell]}|B_{\neq j}) \cdot \mathbf{g}(X, \mathbf{K}, \Gamma, \mathbf{R}) = \mathbf{b}_j,$$

where the polynomials in $\mathbf{g}$ do not depend on $\mathbf{S}$.

Next, we will show that this can only happen if $|\mathsf{Cor}| < t$, which we summarize in the following claim:

Claim 20. *Let $\mathbf{L}_1 \setminus (\mathbf{S}, \mathbf{V})$ and $\mathbf{L}_2 \setminus (\mathbf{S}, \mathbf{V})$ be the elements from the $\mathbf{L}_1$ and $\mathbf{L}_2$, respectively, that do not depend on $\mathbf{S}$ or $\mathbf{V}$.*

$$A = \begin{bmatrix} W(\mathbf{Y}) & 1 & X^N - 1 & X & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & X^2 & 1 & 0 & 0 & 0 \\ 0 & \Gamma & 0 & 0 & 0 & 1 & 0 & 0 \\ X^t & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & X \end{bmatrix},$$

where $W(\mathbf{Y}) = \sum_{i\in\mathcal{H}} K_i L_i(X) + \sum_{i\in\mathsf{Cor}} \mathsf{sk}_i L_i(X)$. Let $A^{[\ell]}$ be the the matrix of dimension $5 \times 8\ell$, where each column of A is repeated ℓ times. Let $\mathbf{b}_i$ be the vector $(0,0,0,0,R_i)^\intercal$. Let $B_{\neq j} = (\mathbf{b}_1|\ldots|\mathbf{b}_j - 1|\mathbf{b}_j + 1|\ldots|\mathbf{b}_\ell)$.

If $|\mathsf{Cor}| < t$ then for all $j \in [\ell]$, there does not symbolically exist any non-zero $\mathbf{g} \in \mathbb{Z}_p[X, \mathbf{K}, \Gamma, \mathbf{R}]^{8\ell+\ell-1}$ with $g_{1,i}, g_{6,i}$, and $g_{8,i}$ coming from $\mathbf{L}_2$, and $g_{2,i}, g_{3,i}, g_{4,i}, g_{5,}, g_{7,i}$ coming from $\mathbf{L}_1$ for all $i \in [\ell]$, and $g_{9,i}$ for $i \neq j$ coming from $\mathbf{L}_T$ such that

$$A^{[\ell]}|B_{\neq j}\mathbf{g} = b_j.$$

Proof of Claim 20. Suppose towards contradiction that $\mathsf{Cor} < t$ and there exists such $\mathbf{g}$ for some $j \in [\ell]$. W.l.o.g., for ease of notation, assume $j = \ell$.

First, we note the equation system.

$$W\bar{g}_1 + \bar{g}_2 + (X^N - 1)\bar{g}_3 + X\bar{g}_4 = 0 \tag{3}$$

$$X^2\bar{g}_4 + \bar{g}_5 = 0 \tag{4}$$

$$\Gamma\bar{g}_2 + \bar{g}_6 = 0 \tag{5}$$

$$X^t\bar{g}_1 + \bar{g}_7 = 0 \tag{6}$$

$$\bar{g}_1 + X\bar{g}_8 + \sum_{i=1}^{\ell-1} g_{9,i} R_i = R_\ell \tag{7}$$

In the above, we denote $\bar{g}_k \coloneqq \sum_{i\in[\ell]} g_{k,i}$.

Next, let us inspect the structure of the different $\bar{g}_k$. For convenience, denote $R_0 \coloneqq 1$. For $k = 2,3,4,5,7$, the function $\bar{g}_k$ is a linear combination from elements of $\mathbf{L}_1 \setminus \mathbf{S}$, thus

$$\bar{g}_k = \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(k)} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}_{u,j,m}^{(k)} K_u X^j \right).$$

Note that this implies that, e.g. for $k = 1$, it holds that $\bar{\alpha}_{j,m}^{(1)} = \sum_{i\in[\ell]} \alpha_{j,m}^{(1,i)}$, and similarly for $\bar{\beta}$. Also, we define

$$\bar{g}_{k,m} = \sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(k)} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}_{u,j,m}^{(k)} K_u X^j,$$

where $\bar{g}_{k,m}$ *does not depend on* any $R_1, \ldots, R_\ell$. As a result, we get

$$\bar{g}_k = \sum_{m=0}^{\ell} \bar{g}_{k,m}.$$

Further, for $k = 1,6,8$, the function $\bar{g}_k$ is a linear combination form elements of $\mathbf{L}_2 \setminus \mathbf{S}$, thus

$$\bar{g}_k = \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(k)} X^j + \bar{\delta}_m^{(k)} \Gamma \right).$$

We define $\bar{g}_{k,m}$ analogously, such that they do not depend on $R_1, \ldots, R_\ell$.

Finally, the $g_{9,i}$ for $1 \leq i < \ell$ are just constants, as they stem from $\mathbf{L}_T$.

In the following steps, we inspect the equations to find constraints on the different $\bar{g}_{k,\ell}$, i.e., the terms that get multiplied by R_ℓ. In the end, we aim to apply the sumcheck lemma on $R_\ell \cdot W \cdot \bar{g}_{1,\ell}$ in Equation 3, and reach a contradiction with the assumption that $|\mathsf{Cor}| < t$.

We start with $\bar{g}_1$.

- Equation 3: Clearly, $R_m\Gamma$ can only come from $\bar{g}_1$, hence we know that $\bar{\delta}_m^{(1)} = 0$ for all $0 \leq m \leq \ell$. Thus

$$\bar{g}_{1,m} = \sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(k)} X^j.$$

- Equation 6: We insert $\bar{g}_1$ and $\bar{g}_7$:

$$0 = X^t \cdot \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(1)} X^j \right) + \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(7)} X^j + \sum_{u \in \mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}_{u,j,m}^{(7)} K_u X^j \right)$$

  By inspection, we can deduce the following constraints:
  - $\bar{\beta}_{u,j,m}^{(7)} = 0$ for all u, j, m, because K_u does not appear elsewhere.
  - $\bar{\alpha}_{j,m}^{(1)} = 0$ for all m, and all $j > N - t$, because the corresponding $\bar{g}_{7,m}$ have maximum degree N in X. Hence, we can write

$$\bar{g}_{1,m} = \sum_{j=0}^{N-t} \bar{\alpha}_{j,m}^{(k)} X^j.$$

- Equation 7: We can rearrange the equation as follows:

$$\bar{g}_1 - \left(R_\ell + \sum_{i=1}^{\ell-1} g_{9,i} R_i \right) = -X \cdot \bar{g}_8$$

$$\sum_{m=0}^{\ell} R_m \bar{g}_{1,m} - \left(R_\ell + \sum_{i=1}^{\ell-1} g_{9,i} R_i \right) = -X \sum_{m=0}^{\ell} R_m \left(\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(8)} X^j + \bar{\delta}_m^{(8)} \Gamma \right)$$

  First, observe that $\bar{\delta}_m^{(8)} = 0$ for all m, because $R_m\Gamma$ does not appear elsewhere. Next, observe that, for $X = 0$, it holds that $\bar{g}_1(X = 0, R_1, \ldots, R_\ell) = R_\ell + \sum_{i=1}^{\ell-1} g_{9,i} R_i$. Because of the term R_ℓ, we can conclude that $\bar{g}_{1,\ell}(0) = 1$, as no factor of R_ℓ appears otherwise in $\bar{g}_1$. Also, recall that $\bar{g}_{1,\ell}$ only depends on the variable X.

Next, we collect constraints on $\bar{g}_2$ using Equation 5.

$$\underbrace{\Gamma \cdot \sum_{m=0}^{\ell} R_m \left(\overbrace{\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(2)} X^j + \sum_{u \in \mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}_{u,j,m}^{(2)} K_u X^j}^{\bar{g}_{2,m}} \right)}_{\bar{g}_2} + \sum_{m=0}^{\ell} R_m \left(\overbrace{\underbrace{\sum_{j=0}^{N} \bar{\alpha}_{j,m}^{(6)} X^j + \bar{\delta}_m^{(6)} \Gamma}_{\bar{g}_{6,m}}}^{\bar{g}_6} \right) = 0$$

First, observe that $\bar{\alpha}_{j,m}^{(6)} = 0$ for all j, m, because all other terms contain a factor of Γ. Thus, it holds that $\bar{g}_{6,m} = \delta_m^{(6)}\Gamma$. Further, it must hold that $\bar{\beta}_{u,j,m}^{(2)} = 0$, because a factor of $\Gamma R_m K_u X^j$ does not appear elsewhere. Finally, it holds that $\bar{\alpha}_{j,m}^{(2)} = 0$ for $j > 0$ and all m, because $R_m\Gamma X$ does not appear elsewhere. We are left with $\bar{g}_{2,m} = \alpha_{0,m}^{(2)}$.

Next, we observe constraints on $\bar{g}_4$ and $\bar{g}_5$ from Equation 4:

$$\bar{g}_5 = -\bar{g}_4 X^2$$

As $\bar{g}_4$ has an additional factor of X^2, it must hold that $\bar{\alpha}_{j,m}^{(5)} = 0$ and $\bar{\beta}_{u,j,m} = 0$ for all u, m, and $j \in \{0, 1\}$.

Finally, we rewrite Equation 3, applying the sum-check lemma to $W \cdot R_\ell \bar{g}_{1,\ell}$. For this, recall that $\bar{g}_{1,\ell} = \sum_{j=0}^{N-t} \bar{\alpha}_{j,\ell}^{(1)} X^j$ and $\bar{g}_{1,\ell}(0) = 1$. We write $\bar{g}_{1,\ell}$ in Lagrange basis form as $\bar{g}_{1,\ell} = \sum_{i=1}^{N} b_i L_i(X)$, where $b_i = \bar{g}_{1,\ell}(\omega^i)$:

$$R_\ell \cdot \left(\sum_{i \in \mathcal{H}} b_i K_i + \sum_{i \in \mathsf{Cor}} b_i \mathsf{sk}_i + Q_x(X) X + Q_Z(X) Z(X) \right) + \sum_{m=0}^{\ell-1} R_m W \bar{g}_{1,m}(X) = -\bar{g}_2 - \bar{g}_3 (X^N - 1) - \bar{g}_4 X$$

We can rewrite this as follows:

$$R_\ell \cdot \left(\overbrace{\sum_{i \in \mathcal{H}} b_i K_i + \sum_{i \in \mathsf{Cor}} b_i \mathsf{sk}_i + Q_x(X) X + Q_Z(X) Z(X)}^{T} \right) + F = R_\ell \left(\underbrace{-\bar{g}_{2,\ell} - \bar{g}_{3,\ell}(X^N - 1) - \bar{g}_{4,\ell} X}_{T'} \right) + F',$$

where both F and F' do not depend on R_ℓ. Hence, it must hold that $T = T'$. We insert $\bar{g}_{2,\ell}, \bar{g}_{3,\ell}$, and replace $X \cdot \bar{g}_{4,\ell}$ with $\bar{g}_{5,\ell}$.

$$\sum_{i\in\mathcal{H}} b_i K_i + \sum_{i\in\mathsf{Cor}} b_i \mathsf{sk}_i + Q_x(X)X + Q_Z(X)(X^N - 1)$$
$$= -\alpha^{(2)}_{0,\ell} -$$
$$\overbrace{\left(\sum_{j=0}^{N} \bar{\alpha}^{(3)}_{j,\ell} X^j + \sum_{u\in\mathcal{H}} \sum_{j=0}^{N-1} \bar{\beta}^{(3)}_{u,j,\ell} K_u X^j\right)}^{\bar{g}_{3,\ell}} (X^N - 1)$$
$$\underbrace{- \left(\sum_{j=2}^{N} \bar{\alpha}^{(5)}_{j,\ell} X^{j-2} + \sum_{u\in\mathcal{H}} \sum_{j=1}^{N-1} \bar{\beta}^{(5)}_{i,j,\ell} K_u X^{j-2}\right) X}_{\bar{g}_{5,\ell}/X}$$

First, observe that it must hold that $Q_Z(X) - \bar{g}_{3,\ell} = 0$, because there does not appear any factor of X^j for $j \geq N$ anywhere else. For a similar reason, it must hold that $Q_x(X) - (\bar{g}_{5,j}/X) = 0$, as there do not appear any other factors of X.

Thus, we can simplify the equation to

$$\sum_{i\in\mathcal{H}} b_i K_i + \sum_{i\in\mathsf{Cor}} b_i \mathsf{sk}_i = -\alpha^{(2)}_{0,\ell}.$$

We can deduce that $b_i = 0$ for all $i \in \mathcal{H}$. Combining with $\bar{g}_{1,\ell}(0) = 1$, we can lower-bound the degree of $\bar{g}_{1,\ell}$ to be $\deg(\bar{g}_{1,\ell}) \geq |\mathcal{H}|$. At the same time, however, we know that $\deg(\bar{g}_{1,\ell}) \leq N - t$, giving us $|\mathcal{H}| \leq N - t$. Since $|\mathcal{H}| = N - |\mathsf{Cor}|$ by definition, we get that $\mathsf{Cor} \geq t$, which is a contradiction to the initial assumption of $\mathsf{Cor} < t$. As a result, the system can not have a symbolic solution g. □

Following the Master Theorem 13, it holds that

$$\Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1] \leq$$
$$\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] + \ell \cdot \frac{(q + v + 2)^2 \cdot (n + 2)}{2^{\lambda+1}},$$

where q is the number of group oracle queries made by the adversary, and v is the number of elements in the lists. Since $\mathcal{A}$ is PPT, the advantage is negligible. □

To close the proof, observe that since $c_{9,i} = [v_i \cdot (f_i + r_i m_i^{(b)})]_1$ for uniform random f_i, it holds that the adversary's view in Game_1 is identically distributed for $b = 0$ and $b = 1$ and thus

$$\Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1] \leq$$
$$\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] + \mathsf{negl}(\lambda) \leq \frac{1}{2} + \mathsf{negl}(\lambda).$$

□

B.2. CCA Security of Σ_{SB}

Lemma 21 (CCA-Security of Σ_{SB})**.** *The Σ_{SB} construction is* SB-IND-CCA*-secure, given the semantic security of Σ_{MSTE}, the pseudorandomness of the* PRF*, the zero-knowledge and simulation-extractability of* $\mathsf{PS}_{\mathsf{cca}}$*, and the zero-knowledge property of* $\mathsf{PS}_{\mathsf{range}}$ *and* $\mathsf{PS}_{\mathsf{pd}}$.

The proof follows the structure of the CCA-security proof in [18] with some extra steps. In particular, we replace the Pedersen commitments $\{\mathsf{com}_i\}_{i\in[\ell]}$ to k_i with commitments to 0 in the challenge ciphertext at the appropriate time. Also, we now simulate partial decryption by extracting s_3 from $\mathsf{PS}_{\mathsf{cca}}$. For the sake of completeness, we next present the entire CCA-security proof.

Proof of Lemma 21. We prove SB-IND-CCA security of Σ_{SB}(Definition 7) through a series of game-hops where we eliminate all dependencies on the internal bit b of Game-SB-IND-CCA. Let $\mathsf{Game}_{0,\mathcal{A}}$ be the original game defined in Figure 1, instantiated with Σ_{SB}.

$\mathsf{Game}_{1,\mathcal{A}}$: In this game, we change how the proof π^{cca} is computed by the game during encryption of the challenge m_b. Instead of computing π^{cca} as $\mathsf{PS}_{\mathsf{cca}}.\mathsf{Prove}(\chi, \omega)$, where $\chi = (\mathsf{crs}, \{\mathsf{pk}_i\}_{[n]}, \mathsf{ct}, \pi^{\mathsf{range}})$ and $\omega = (k_1, \ldots, k_\ell, \vec{s})$, we simulate the proof using the simulator Sim of $\mathsf{PS}_{\mathsf{cca}}$. Hence, we set $\pi^{\mathsf{cca}} \leftarrow\$ \mathsf{PS}_{\mathsf{cca}}.\mathsf{Sim}(\mathsf{crs}, \{\mathsf{pk}_i\}_{[n]}, \mathsf{ct}, \pi^{\mathsf{range}})$.

Claim 22. *If the proof system $\mathsf{PS}_{\mathsf{cca}}$ is zero-knowledge (Definition A.3) then $\mathsf{Game}_{1,\mathcal{A}}$ is computationally indistinguishable from $\mathsf{Game}_{0,\mathcal{A}}$.*

$$\left|\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda) = 1]\right| \leq \mathsf{negl}(\lambda)$$

Proof of Claim 22. The claim follows straight from the zero-knowledge property of the proof system $\mathsf{PS}_{\mathsf{cca}}$. □

$\mathsf{Game}_{2,\mathcal{A}}$: In this game, we now simulate the proof π^{range} in a similar manner. To do so, we set $\pi^{\mathsf{range}} \leftarrow\$ \mathsf{PS}_{\mathsf{range}}.\mathsf{Sim}(\mathsf{crs}, \{\mathsf{com}_i\}_{[n]})$ for the challenge ciphertext.

Claim 23. *If the proof system $\mathsf{PS}_{\mathsf{range}}$ is zero-knowledge (Definition A.3) then $\mathsf{Game}_{2,\mathcal{A}}$ is computationally indistinguishable from $\mathsf{Game}_{1,\mathcal{A}}$.*

$$\left|\Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) = 1]\right| \leq \mathsf{negl}(\lambda)$$

Proof of Claim 23. The claim follows straight from the zero-knowledge property of the proof system $\mathsf{PS}_{\mathsf{range}}$. □

$\mathsf{Game}_{3,\mathcal{A}}$: In this game, we change how the Pedersen commitments $\{\mathsf{com}_i\}_{i\in[\ell]}$ are computed for the challenge ciphertext. Instead of committing with $\mathsf{com}_i \leftarrow\!\$\ \mathsf{Ped.Commit}(k_i)$ to each k_i, we instead set $\mathsf{com}_i \leftarrow\!\$\ \mathsf{Ped.Commit}(1)$ for all $i \in [\ell]$.

Claim 24. *For all (even unbounded) adversaries $\mathcal{A}$ it holds that*

$$\Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda) = 1] = \Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda) = 1].$$

Proof of Claim 24. The claim follows directly from the perfect hiding property of the Pedersen commitment scheme. □

In a following game-hop, we want to make a reduction to the semantic security of the underlying Σ_{MSTE} scheme (Definition 18). In order to do that, we need to simulate the batch-decryption oracle $\mathcal{O}^{\text{b-dec}}$, which has a dependency on the threshold silent setup secret keys sk_i for $i \in \mathcal{H}$. Because we only require Σ_{MSTE} to be semantically secure, we do not have access to a decryption oracle, which we could use to simulate the partial decryptions of Σ_{MSTE}. Hence, we remove the dependency on sk_i of $\mathcal{O}^{\text{b-dec}}$ within the following three game-hops using the zero-knowledge property of the proof system $\mathsf{PS}_{\mathsf{pd}}$, and simulation-extractability of the proof system $\mathsf{PS}_{\mathsf{cca}}$ to simulate decryption shares.

$\mathsf{Game}_{4,\mathcal{A}}$: In this game, we replace the proof of valid partial decryption π^{pd} that is generated within $\mathcal{O}^{\text{b-dec}}$ with a simulated one.

$$\pi^{\mathsf{pd}} \leftarrow\!\$\ \mathsf{PS}_{\mathsf{pd}}.\mathsf{Sim}(\mathsf{td}, (\mathsf{pp}, \mathsf{pk}_i, \{\mathsf{ct}_i\}_{i\in[B]}, \mathsf{pd}))$$

We call the resulting oracle $\mathcal{O}_4^{\text{b-dec}}$

Claim 25. *If the proof system $\mathsf{PS}_{\mathsf{pd}}$ is zero-knowledge (Definition A.3) then $\mathsf{Game}_{4,\mathcal{A}}$ is computationally indistinguishable from $\mathsf{Game}_{3,\mathcal{A}}$.*

$$\left|\Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda) = 1]\right| \leq q \cdot \mathsf{negl}(\lambda),$$

where q denotes the number of $\mathcal{O}^{\text{b-dec}}$ queries made by $\mathcal{A}$.

Proof of Claim 25. The claim follows directly from the zero-knowledge property of the proof system $\mathsf{PS}_{\mathsf{pd}}$. □

$\mathsf{Game}_{5,\mathcal{A}}$: In this game, we make changes to the batch-decryption oracle $\mathcal{O}^{\text{b-dec}}$. For each ciphertext in the batch ct_j with $j \in [B]$ we do the following: After verifying the proof π_j^{cca}, we extract the witnesses $\omega_j = (\vec{\tilde{k}}_j, \vec{\tilde{s}}_j, \vec{\tilde{r}}_j)$, where $\vec{\tilde{k}}_j$ corresponds to the chunk keys of ciphertext j, $\vec{\tilde{s}}_j$ corresponds to the randomness during encryption in Σ_{MSTE} for ciphertext j, and $\vec{\tilde{r}}_j$ corresponds to the randomness used for the Pedersen commitments in ciphertext j. Next, we assert the validity of the extraction, i.e., we check that

$$\mathsf{ct}_j = (j, t, \mathsf{PRF.Punct}(\vec{\tilde{k}}_j, j), \beta, \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}(\{\mathsf{pk}_i\}_{i\in[n]}, \vec{\tilde{k}}_j; \vec{\tilde{s}}_j, t), \mathsf{Ped.Commit}(\vec{\tilde{k}}_j; \vec{\tilde{r}}_j)).$$

We abort if the check fails. We call the resulting oracle $\mathcal{O}_5^{\text{b-dec}}$.

Claim 26. *If the underlying proof system $\mathsf{PS}_{\mathsf{cca}}$ is simulation-extractable (Definition A.3) then $\mathsf{Game}_{5,\mathcal{A}}$ is computationally indistinguishable from $\mathsf{Game}_{4,\mathcal{A}}$.*

$$\left|\Pr[\mathsf{Game}_{5,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) = 1]\right| \leq \mathsf{negl}(\lambda)$$

Proof of Claim 26. Clearly, the only way for a PPT adversary $\mathcal{A}$ to distinguish between the games is by triggering the additional abort conditions in $\mathcal{O}_5^{\text{b-dec}}$. To do so, $\mathcal{A}$ must submit a pair $(\mathsf{ct}_j, \pi_j^{\mathsf{cca}})$, where c_j is not the challenge ciphertext with a verifying proof π_j^{cca} such that the extractor $\mathsf{PS}_{\mathsf{cca}}.\mathsf{Extract}$ fails to extract a correct witness ω. The proof for the challenge ciphertext $\mathsf{ct}^\star$ can be simulated using the SimProve oracle. Because the oracle would abort anyway if $\mathsf{ct}_j = \mathsf{ct}^\star$, it holds that $\chi = (\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in[n]}, \mathsf{ct}, \pi^{\mathsf{range}}) \notin \mathcal{Q}$, hence the simulation-extractability of $\mathsf{PS}_{\mathsf{cca}}$ implies that $\mathcal{A}$'s probability of success is negligible. Given an adversary $\mathcal{A}$ that submits a total of q ciphertexts for decryption to $\mathcal{O}^{\text{b-dec}}$, we get

$$\left|\Pr[\mathsf{Game}_{5,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) = 1]\right| \leq q \cdot \mathsf{negl}(\lambda)$$

which is negligible because q is bounded by a polynomial in λ. □

$\mathsf{Game}_{6,\mathcal{A}}$: In this game we make another change to $\mathcal{O}^{\text{b-dec}}$, which finally removes its dependency on sk_i for $i \in \mathcal{H}$. Instead of computing partial decryption in Σ_{MSTE} on the aggregated ciphertext[8] $\mathsf{ct} = \sum_{j\in[B]} \mathsf{ct}_j$ using sk_i, we simulate partial decryption using the extracted $\{\tilde{s_3}\}_{j\in[B]}$. In particular,

8. Here we abuse notation about and define aggregation of ciphertexts in a batch through the $\sum$ operator.

we compute the partial decryption as follows: First, set $\tilde{\mathbf{s_3}} = \sum_{j\in[B]} \tilde{s_{3j}}$. Then, compute

$$\mathsf{pd}_i \leftarrow \tilde{\mathbf{s_3}} \cdot \mathsf{pk}_{0,0}^{(i)}\big[= [\mathsf{sk}_i \cdot \tilde{\mathbf{s_3}}]_1\big],$$

which can be computed without knowledge of sk_i. We denote the resulting oracle as $\mathcal{O}_6^{\text{b-dec}}$

Claim 27. *The games are identical, thus*

$$\Pr[\mathsf{Game}_{6,\mathcal{A}}(1^\lambda) = 1] = \Pr[\mathsf{Game}_{5,\mathcal{A}}(1^\lambda) = 1].$$

Proof of Claim 27. The partial decryption as computed from the secret key (Game_5) can only differ from the one computed using $\tilde{\mathbf{s_3}}$ (Game_6), if there exists at least one $j \in [B]$ such that $[\tilde{s_{3j}}]_1 \neq \mathsf{ct}_6^{(j)}$. In this case, however, both games would have aborted due to the abort condition that was first introduced in the previous game hop (Game_5), leading both oracles to return $\perp$ instead. □

Now that we have removed the dependency on the threshold silent setup secret keys sk_i from the batch-decryption oracle $\mathcal{O}_6^{\text{b-dec}}$, we can proceed with a game hop that replaces encryption of $([k_1]_T, \ldots, [k_\ell]_T)$ in the challenge ciphertext with an encryption of an arbitrary constant (say $([1]_T, \ldots, [1]_T)$. We can reduce the indistinguishability of this game hop to the semantic security of Σ_{MSTE}. We do not run into issues with simulating $\mathcal{O}^{\text{b-dec}}$ queries in this reduction, because we no longer need the secret keys to do so.

$\mathsf{Game}_{7,\mathcal{A}}$: In this game we change how the challenge ciphertext is computed and replace the encryption of $([k_1]_T, \ldots, [k_\ell]_T)$ with an encryption of $([1]_T, \ldots, [1]_T)$ (of length ℓ). Instead of computing $\mathsf{ct} \leftarrow\$\ \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}(\{\mathsf{pk}_i\}_{[n]}, \vec{k})$, we set $\mathsf{ct} \leftarrow\$\ \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}(\{\mathsf{pk}_i\}_{[n]}, \vec{1})$.

Claim 28. *If the underlying Σ_{MSTE} scheme is semantically secure (Definition 18), then $\mathsf{Game}_{7,\mathcal{A}}$ is computationally indistinguishable from $\mathsf{Game}_{6,\mathcal{A}}$.*

$$\big|\Pr[\mathsf{Game}_{7,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{6,\mathcal{A}}(1^\lambda) = 1]\big| \leq \mathsf{negl}(\lambda)$$

Proof of Claim 28. We prove this claim by reduction to the semantic security of Σ_{MSTE}. Let $\mathcal{A}$ be a PPT adversary such that

$$\big|\Pr[\mathsf{Game}_{7,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathsf{Game}_{6,\mathcal{A}}(1^\lambda) = 1]\big| \geq \epsilon(\lambda)$$

for a non-negligible ϵ. We construct a PPT reduction $\mathcal{B}$ that runs in $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ (Figure 1)and internally uses $\mathcal{A}$ to win.

$\mathcal{B}$ receives crs from $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ and runs $\mathcal{A}$ with (crs) as input, receiving a universe $\mathcal{C} \subseteq [n]$ and a set of corrupted parties $\mathsf{Cor} \subseteq \mathcal{C}$, which $\mathcal{B}$ forwards to $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$, receiving a set of public keys $\{\mathsf{pk}_i\}_{i\in\mathcal{H}}$. $\mathcal{B}$ sends the honest parties' public keys to $\mathcal{A}$ and receives a set of public keys for the corrupted parties $\{\mathsf{pk}_i\}_{i\in\mathsf{Cor}}$. It follows the (identical) steps from Game_6 and Game_7 up to the point where it is supposed to encrypt $\vec{k} := ([k_1]_T, \ldots, [k_\ell]_T)$ or $\vec{1} := ([1]_T, \ldots, [1]_T)$ respectively as part of the challenge ciphertext. Instead, $\mathcal{B}$ sends

$$(m_0 := \vec{k}, m_1 := \vec{1})$$

to $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ and receives ct as a response. $\mathcal{B}$ then continues following the steps from Game_6 and Game_7.

Note that $\mathcal{B}$ can simulate the batch decryption oracle $\mathcal{O}_7^{\text{b-dec}} = \mathcal{O}_6^{\text{b-dec}}$ to $\mathcal{A}$, as it no longer depends on any unknown secret key share sk_i for $i \in \mathcal{H}$. This dependency was removed in the previous game hop.

Analysis.. Let b' be the internal bit of $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$. If $b' = 0$, then $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ returns an encryption of $\vec{k}$ and $\mathcal{B}$ simulates $\mathsf{Game}_{5,\mathcal{A}}$ to $\mathcal{A}$. If $b' = 1$, then $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ returns an encryption of $\vec{1}$ and $\mathcal{B}$ simulates $\mathsf{Game}_{6,\mathcal{A}}$ to $\mathcal{A}$. Hence, $\mathcal{B}$ wins $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ if $\mathcal{A}$ wins the distinguishing game, which is assumed to be non-negligible. Denote $\mathcal{B}$'s output in $\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}$ with d.

$$\begin{aligned}
&\left|\Pr\left[\mathsf{Game\text{-}S\text{-}IND\text{-}CPA}_{\mathcal{B}}^{\Sigma_{\mathsf{MSTE}}}(1^\lambda) = 1\right]\right| \\
&= \left|\frac{1}{2}\Pr[d = 0 \mid b' = 0] + \frac{1}{2}\Pr[d = 1 \mid b' = 1]\right| \\
&= \left|\frac{1}{2}(1 - \Pr[d = 1 \mid b' = 0] + \Pr[d = 1 \mid b' = 1])\right| \\
&= \frac{1}{2} + \frac{1}{2}\big|- \Pr[\mathsf{Game}_{6,\mathcal{A}}(1^\lambda) = 1] \\
&+ \Pr[\mathsf{Game}_{7,\mathcal{A}}(1^\lambda) = 1]\big| \\
&> \frac{1}{2} + \frac{\epsilon(\lambda)}{2}
\end{aligned}$$

This contradicts the semantic security of Σ_{MSTE}. □

$\mathsf{Game}_{8,\mathcal{A}}$: In this game, we replace the result of the PRF-evaluation during encryption with a random group element from $\mathbb{G}_T$. Instead of computing $\beta \leftarrow m_b + \mathsf{PRF}(k, i)$ we set $\beta \leftarrow m_b + r$ for a random $r \leftarrow\$\ \mathbb{G}_T$.

Claim 29. *If* PRF *is pseudorandom (Definition 17) then* $\mathsf{Game}_{8,\mathcal{A}}$ *is computationally indistinguishable from* $\mathsf{Game}_{7,\mathcal{A}}$.

$$\left|\Pr[\mathsf{Game}_{8,\mathcal{A}}(1^\lambda)=1]-\Pr[\mathsf{Game}_{7,\mathcal{A}}(1^\lambda)=1]\right| \leq \mathsf{negl}(\lambda)$$

Proof Sketch of Claim 29. We prove this claim by reduction to the pseudorandomness Game-Pr of PRF. The PPT reduction $\mathcal{B}$ playing in Game-$\mathsf{Pr}_{\mathcal{B}}$ sends index i to the pseudorandomness game and receives k' and value r as a response. $\mathcal{B}$ then computes β as $\beta \leftarrow m_b + r$. If the r is sampled uniformly random by the pseudorandomness game ($b=0$), $\mathcal{B}$ simulates $\mathsf{Game}_{8,\mathcal{A}}$ to $\mathcal{A}$. If r is the result of the PRF evaluation ($b=1$), $\mathcal{B}$ simulates $\mathsf{Game}_{7,\mathcal{A}}$ to $\mathcal{A}$. Hence, if $\mathcal{A}$ can distinguish between $\mathsf{Game}_{7,\mathcal{A}}$ and $\mathsf{Game}_{8,\mathcal{A}}$, then $\mathcal{B}$ can distinguish between $b=0$ and $b=1$. □

$\underline{\mathsf{Game}_{9,\mathcal{A}}}$: In this game we directly sample β from $\mathbb{G}_T$ instead of computing it as $m_b + r$ for a random $r \leftarrow\!\$\, \mathbb{G}_T$.

Claim 30. *Both games are identically distributed, and thus it holds that*

$$\Pr\left[\mathsf{Game}_{9,\mathcal{A}}(1^\lambda)=1\right] = \Pr\left[\mathsf{Game}_{8,\mathcal{A}}(1^\lambda)=1\right]$$

Proof of Claim 30. Clearly, β is identically distributed in both games. □

We conclude the proof by arguing that the adversary's view in $\mathsf{Game}_{9,\mathcal{A}}$ no longer depends on the internal bit b. Hence $\Pr\left[\mathsf{Game}_{9,\mathcal{A}}(1^\lambda)=1\right] = \frac{1}{2}$. Further, we have derived $\mathsf{Game}_{9,\mathcal{A}}$ from Game-SB-IND-CCA$^{\Sigma_{\mathsf{SB}}}_{\mathcal{A}}$ through a series of game-hops where each game is computationally indistinguishable from the previous one. We conclude that for all PPT adversaries $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\lambda)$ such that

$$\begin{aligned}&\Pr[\text{Game-SB-IND-CCA}^{\Sigma_{\mathsf{SB}}}_{\mathcal{A}}(1^\lambda)=1]\\ &\leq \Pr[\mathsf{Game}_{9,\mathcal{A}}(1^\lambda)=1]+\mathsf{negl}(\lambda)=\frac{1}{2}+\mathsf{negl}(\lambda)\end{aligned}$$

which satisfies Definition 7. □

B.3. Rogue Ciphertext Security of Σ_{SB}

Lemma 31 (Rogue Ciphertext Security of Σ_{SB})**.** *The* Σ_{SB} *construction is secure against rogue ciphertext attacks given the correctness of the* PRF *and* Σ_{MSTE}*, and the knowledge-soundness of* $\mathsf{PS}_{\mathsf{cca}}$*,* $\mathsf{PS}_{\mathsf{range}}$*, and* $\mathsf{PS}_{\mathsf{pd}}$.

Proof of Lemma 31. The rogue ciphertext security of Σ_{SB} follows from the soundness of the proof system $\mathsf{PS}_{\mathsf{KGen}}$, the knowledge-soundness of the proof systems $\mathsf{PS}_{\mathsf{cca}}$, $\mathsf{PS}_{\mathsf{pd}}$, and $\mathsf{PS}_{\mathsf{range}}$, and the computational binding property of the Pedersen commitment scheme Ped. To summarize, in Game-Rogue, the adversary can control the keys of all parties in the universe and pick some index i and a message m_i. The challenger then encrypts a challenge ciphertext ct_i. The adversary can generate up to $B_{\max}-1$ additional ciphertexts $\{\mathsf{ct}_j, \pi^{\mathsf{ct}}_j\}_{j\in[B_{\max}],j\neq i}$. Also, the adversary generates at least t partial decryptions $\{\mathsf{pd}_u, \pi^{\mathsf{pd}}_u\}_{u\in S}$ on the batch, including the challenge ciphertext at position i. The adversary wins if all proofs π^{ct}_j and π^{pd}_u verify and the decryption for index i returns a message $m'_i \neq m_i$.

First, $\mathsf{PS}_{\mathsf{KGen}}$ ensures the validity of the public keys chosen by the adversary, i.e., it must hold except with negligible probability that $\mathsf{pk}_u \in \mathsf{KGen}(1^\lambda)$ for all $u \in \mathcal{C}$.

Let $\mathsf{Game}_0 = \text{Game-Rogue}^{\Sigma_{\mathsf{SB}}}_{\mathcal{A}}$. We make a series of modifications to Game_0, showing that $\mathcal{A}$'s success probability increases at most by a negligible amount.

$\underline{\mathsf{Game}_1}$. In Game_1, we extract the witnesses ω^{cca}_j from $(\mathsf{ct}_j, \pi^{\mathsf{ct}}_j)$ using the extractor

$$\omega^{\mathsf{cca}}_j \leftarrow\!\$\, \mathsf{PS}_{\mathsf{cca}}.\mathsf{Ext}(\mathsf{td}, (\mathsf{crs}, \mathsf{ek}, \mathsf{ct}_j), \pi^{\mathsf{ct}}_j),$$

where $\mathsf{ct}_j = (j, t, k^*, \beta, \mathsf{ct}_{\Sigma_{\mathsf{MSTE}}}, \mathsf{com}_1, \ldots \mathsf{com}_\ell, \pi^{\mathsf{range}})$. We parse ω^{cca}_j as $((k^{\mathsf{cca}}_1, \ldots, k^{\mathsf{cca}}_\ell), \vec{s}, (r^{\mathsf{com}}_1, \ldots, r^{\mathsf{com}}_\ell))$, omitting the index j for readability. We add an additional abort condition and return 0, if for any $j \in [B]$, one of the following conditions is met:

- $k^* \neq \mathsf{PRF}.\mathsf{Punct}\left(\sigma_{i\in[\ell]} k^{\mathsf{cca}}_i \cdot 2^{i-1}, j\right)$
- $\mathsf{ct}_{\mathsf{MSTE}} \neq \Sigma_{\mathsf{MSTE}}.\mathsf{Enc}(\mathsf{ek}, \{k^{\mathsf{cca}}_i\}_{i\in[\ell]}; \vec{s})$
- $\exists i \in [\ell]\colon \mathsf{com}_i \neq \mathsf{Ped}.\mathsf{Commit}(k^{\mathsf{cca}}_i, r^{\mathsf{com}}_i)$

Claim 32. *For all PPT adversaries* $\mathcal{A}$*, it holds that*

$$\begin{aligned}&\Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda) - \Pr[\mathsf{Game}_{0,\mathcal{A}}(1^\lambda)]]\\ &\leq B_{\max} \cdot \mathsf{Adv}^{\textit{know.-sound.}}_{\mathsf{PS}_{\mathsf{cca}},\mathcal{A}}(1^\lambda).\end{aligned}$$

Proof of Claim 32. The claim follows a straightforward reduction to knowledge-soundness of $\mathsf{PS}_{\mathsf{cca}}$. □

$\underline{\mathsf{Game}_2}$: In the next game, we extract a witness $\omega^{\mathsf{range}}_j$ from $(\mathsf{ct}_j, \pi^{\mathsf{range}}_j)$ as

$$\omega^{\mathsf{range}}_j \leftarrow\!\$\, \mathsf{PS}_{\mathsf{range}}.\mathsf{Ext}(\mathsf{td}, (\mathsf{crs}, \{\mathsf{com}_i\}_{i\in[\ell]}, \pi^{\mathsf{range}}_j).$$

We parse $\omega^{\mathsf{range}}_j := (k^{\mathsf{range}}_1, \ldots, k^{\mathsf{range}}_\ell, r^{\mathsf{com}}_1, r^{\mathsf{com}}_\ell)$, omitting the index j for readability. We add an additional abort condition and return 0 if one of the following conditions is met for any j:

- $\exists i \in [\ell]\colon \mathsf{com}_i \neq \mathsf{Ped.Commit}(k_i^{\mathsf{range}}, r_i^{\mathsf{com}})$
- $\exists i \in [\ell]\colon k_i^{\mathsf{range}} \notin [0, 2^z - 1]$, where $z = \lambda/\ell$.

Claim 33. *For all PPT adversaries $\mathcal{A}$, it holds that*

$$\Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda) - \Pr[\mathsf{Game}_{1,\mathcal{A}}(1^\lambda)]] \leq B_{\max} \cdot \mathsf{Adv}^{\textit{know.-sound.}}_{\mathsf{PS}_{\mathsf{range}},\mathcal{A}}(1^\lambda).$$

Proof of Claim 33. The claim follows a straightforward reduction to knowledge-soundness of $\mathsf{PS}_{\mathsf{range}}$. □

$\underline{\mathsf{Game}_3}$: In Game_3, we add an additional abort condition. In particular, we return 0 if for any j, in ω_j^{cca} and $\omega_j^{\mathsf{range}}$, there exists an $i \in [\ell]$ such that $k_i^{\mathsf{cca}} \neq k_i^{\mathsf{range}}$.

Claim 34. *For all PPT adversaries $\mathcal{A}$, there exists a PPT adversary $\mathcal{B}$ such that*

$$\Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda)] - \Pr[\mathsf{Game}_{2,\mathcal{A}}(1^\lambda)] \leq \mathsf{Adv}^{\textit{binding}}_{\mathsf{Ped},\mathcal{B}}(1^\lambda).$$

Proof of Claim 34. The claim follows by reduction to the computational binding property of the Pedersen commitment scheme Ped. Clearly, a PPT adversary $\mathcal{A}$ can only distinguish by triggering the additional abort condition. In this case, however, it holds for some $j \in [B_{\max}]$ there exists and $i \in [\ell]$ such that $\mathsf{Ped.Commit}(k_i^{\mathsf{cca}}, r_i^{\mathsf{com,cca}})$, and $\mathsf{Ped.Commit}(k_i^{\mathsf{range}}, r_i^{\mathsf{com,range}})$, but $k_i^{\mathsf{cca}} \neq k_i^{\mathsf{range}}$. The reduction submits the tuple $((k_i^{\mathsf{cca}}, r_i^{\mathsf{com,cca}}), (k_i^{\mathsf{range}}, r_i^{\mathsf{com,range}}))$ to the binding game and wins, whenever $\mathcal{A}$ hits the additional abort condition. □

$\underline{\mathsf{Game}_4}$: In Game_4, we extract a witness ω_u from all $(\mathsf{pd}_u, \pi_u^{\mathsf{pd}})$ for $u \in S$ using the extractor $\omega_u \leftarrow\!\!\$\ \mathsf{PS}_{\mathsf{pd}}.\mathsf{Ext}(\mathsf{td}, (\mathsf{crs}, \mathsf{pk}_u, \{\mathsf{ct}_j\}_{j\in[\ell]}, \mathsf{pd}_u), \pi_u^{\mathsf{pd}})$. We parse ω_u as (sk_u). We add an additional abort condition and return 0, if $(\mathsf{pd}_u, \_) \neq \mathsf{BatchDec}(\mathsf{sk}_u, \{\mathsf{ct}_j\}_{j\in[B]})$.

Claim 35. *For all PPT adversaries $\mathcal{A}$, it holds that*

$$\Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda) - \Pr[\mathsf{Game}_{3,\mathcal{A}}(1^\lambda)]] \leq N \cdot \mathsf{Adv}^{\textit{know.-sound.}}_{\mathsf{PS}_{\mathsf{pd}},\mathcal{A}}(1^\lambda).$$

Proof of Claim 35. The claim follows a straightforward reduction to knowledge-soundness of $\mathsf{PS}_{\mathsf{pd}}$. □

Because of (a) the perfect correctness of the PRF, (b) the perfect correctness of Σ_{MSTE}, and (c) the perfect correctness and efficiency of the dlog extraction algorithm if $k \in [0, 2^{z+\log(B)} - 1]$, it can only happen that $m_i \neq m_i'$ if one of the following conditions is violated:

1) $\exists \mathsf{pk}_{u'} \in \{\mathsf{pk}_u\}_{u\in\mathcal{C}}$ such that $\mathsf{pk}_{u'} \notin \mathsf{KGen}(1^\lambda)$. This is impossible due to the perfect soundness of KVfy.
2) $\exists j' \in [B]$ such that $k^*_{j'}$ and $\mathsf{ct}^{(j')}_{\Sigma_{\mathsf{MSTE}}}$ do not belong to the same chunking $(k_1, \ldots, k_\ell)$ or there exists at least one k_i such that $k_i \notin [0, 2^z - 1]$.
3) $\exists u \in S$ such that pd_u is not a valid partial decryption of the batch of ciphertexts.

Since conditions 2 and 3 cause the adversary to trigger an abort condition in Game_4, it holds that

$$\Pr[\mathsf{Game}_{4,\mathcal{A}}(1^\lambda)] = 0.$$

Thus, we can bound the advantage of any PPT adversary $\mathcal{A}$ to execute a successful rogue ciphertext attack as follows:

$$\Pr[\mathsf{Game\text{-}Rogue}^{\Sigma_{\mathsf{SB}}}_{\mathcal{A}}(1^\lambda) = 1] \leq B_{\max} \cdot \left(\mathsf{Adv}^{\text{know.-sound.}}_{\mathsf{PS}_{\mathsf{cca}},\mathcal{A}}(1^\lambda) + \mathsf{Adv}^{\text{know.-sound.}}_{\mathsf{PS}_{\mathsf{range}},\mathcal{A}}(1^\lambda)\right) + N \cdot \mathsf{Adv}^{\text{know.-sound.}}_{\mathsf{PS}_{\mathsf{pd}},\mathcal{A}}(1^\lambda).$$

□